

Module: 3

Microcontroller Architecture: Intel 8051

Dr. Sukanta Kumar Tulo

VIT Vellore

Microcontroller Architecture: Intel 8051

- 1. Microcontroller 8051 - Organization and Architecture,**
- 2. RAM-ROM Organization, Machine Cycle,**
- 3. Instruction set: Addressing modes,**
- 4. Data Processing - Stack, Arithmetic, Logical;**
- 5. Branching – Unconditional and Conditional,**
- 6. Assembly programming.**

Intel 8051 Microcontroller (Also known as MCS-51)

- Intel 8051 Microcontroller (Also known as MCS-51) is a Harvard architecture, Complex Instruction Set Computing (CISC) Architecture, single chip microcontroller (μ C) series developed by Intel in 1980 for use in embedded systems
- Intel's original MCS-51 family was developed using N-type metal oxide-semiconductor (NMOS) technology
- But later versions, identified by a letter C in their name (e.g., 80C51) used Complementary Metal–oxide–semiconductor (CMOS) technology and consume less power

Features of 8051 μ C

- 8051 is complete computer system built on one Chip.
- 40 Pin IC
- It has CPU, RAM, ROM, Serial port, Parallel Port, Interrupt, Timer on single chip
- 5V power supply
- 8051 operates at 12MHz clock frequency
- 8051 has 8 bit of ALU.
- 8051 has 8 bits of Data bus
- 8051 Follows **Harvard architecture**
- CISC Machine
- 8051 has 4KB of internal ROM for program storage.
- 8051 has 128 bytes of RAM for data storage.

Harvard Architecture : it has separate memory for program and data storage

Features of 8051 μ C

- 8051 has four 8 bits of IO ports
- 8051 has a serial port synchronous and asynchronous communication
- 8051 has two 16 bits UP timers
- 8051 has five interrupts operating at priority levels
- 8051 has two power saving modes. Idle mode and Power down mode
- 8051 has 16 bits of address bus which can be used to interface external memory of RAM or ROM.

This ports can be used for interfacing of peripherals like keyboard display, Stepper motor, LEDs, and switches etc

8051 Features

- **8-bit Microcontroller**
- 4 KB Internal ROM
- 128 Bytes Internal RAM
- Four 8 BIT I/O PORTS (32 I/O LINES)
- Two 16 Bit Timers/Counters
- On Chip Full Duplex UART for Serial Communication
- 8-bit bidirectional data bus
- 16-bit unidirectional address bus
- 64k External Code Memory

8051 Features

- 5 Vector Interrupts (2 External, 3 Internal - Timer0, Timer1, Serial)
- On Chip Clock Oscillator with 12 MHz clock frequency
- 16-bit Program Counter (PC) to access external Code Memory
- 16-bit Data Pointer (DP) to access external Data Memory
- 128 user defined flags
- 32 General Purpose Registers each of 8 bits
- **21 special function registers SFR**
- Crystal Frequency of 11.0592MHz $\approx$ 12MHz
- It support many special features like UARTs, ADC/DAC, Op-amp etc.

8051 Family

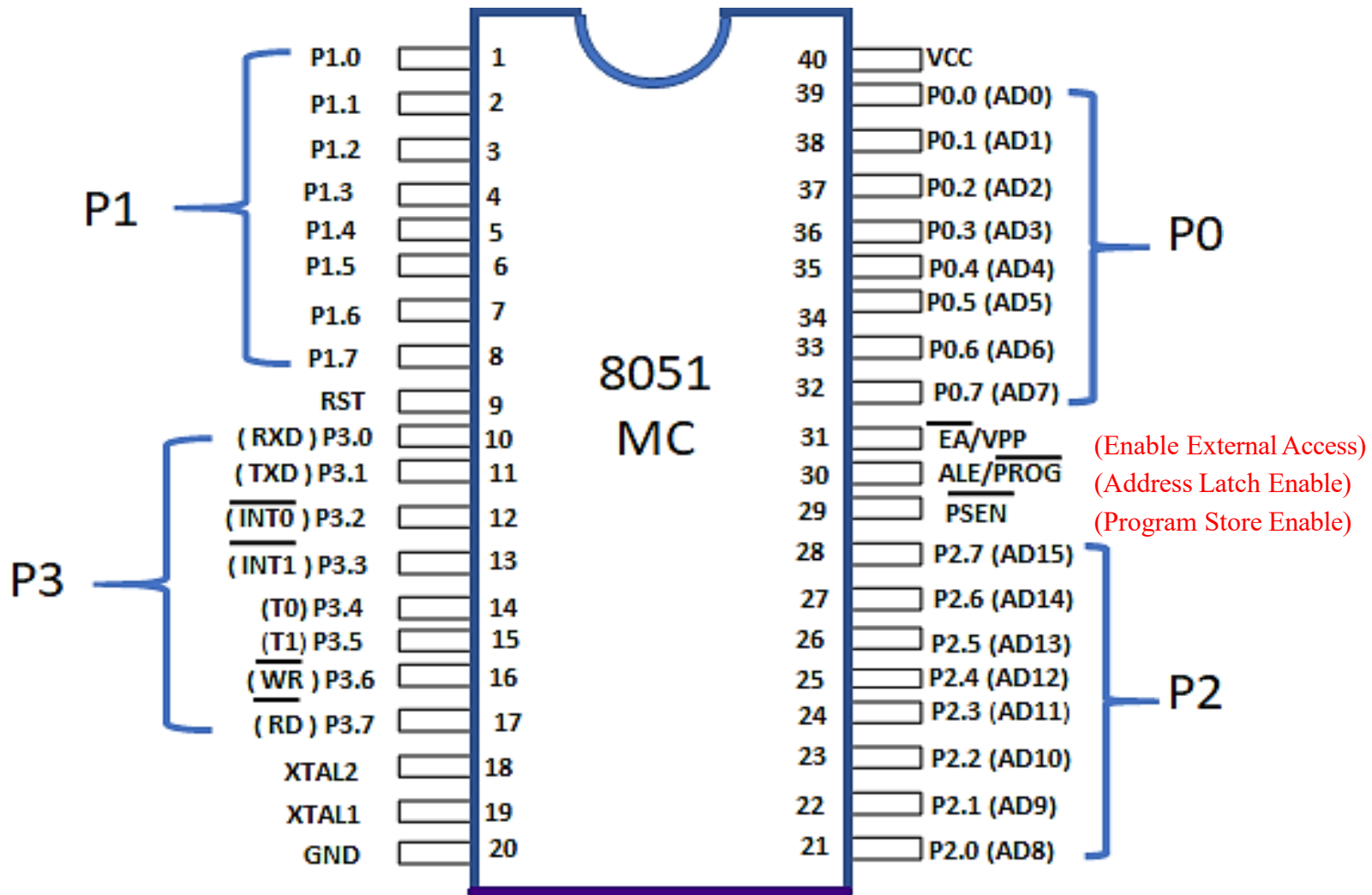
Feature	8051	8052	8031
ROM (on-chip program space in bytes)	4K	8K	0K
RAM (bytes)	128	256	128
Timers	2	3	2
I/O pins	32	32	32
Serial port	1	1	1
Interrupt sources	6	8	6

8051 Pin Diagram

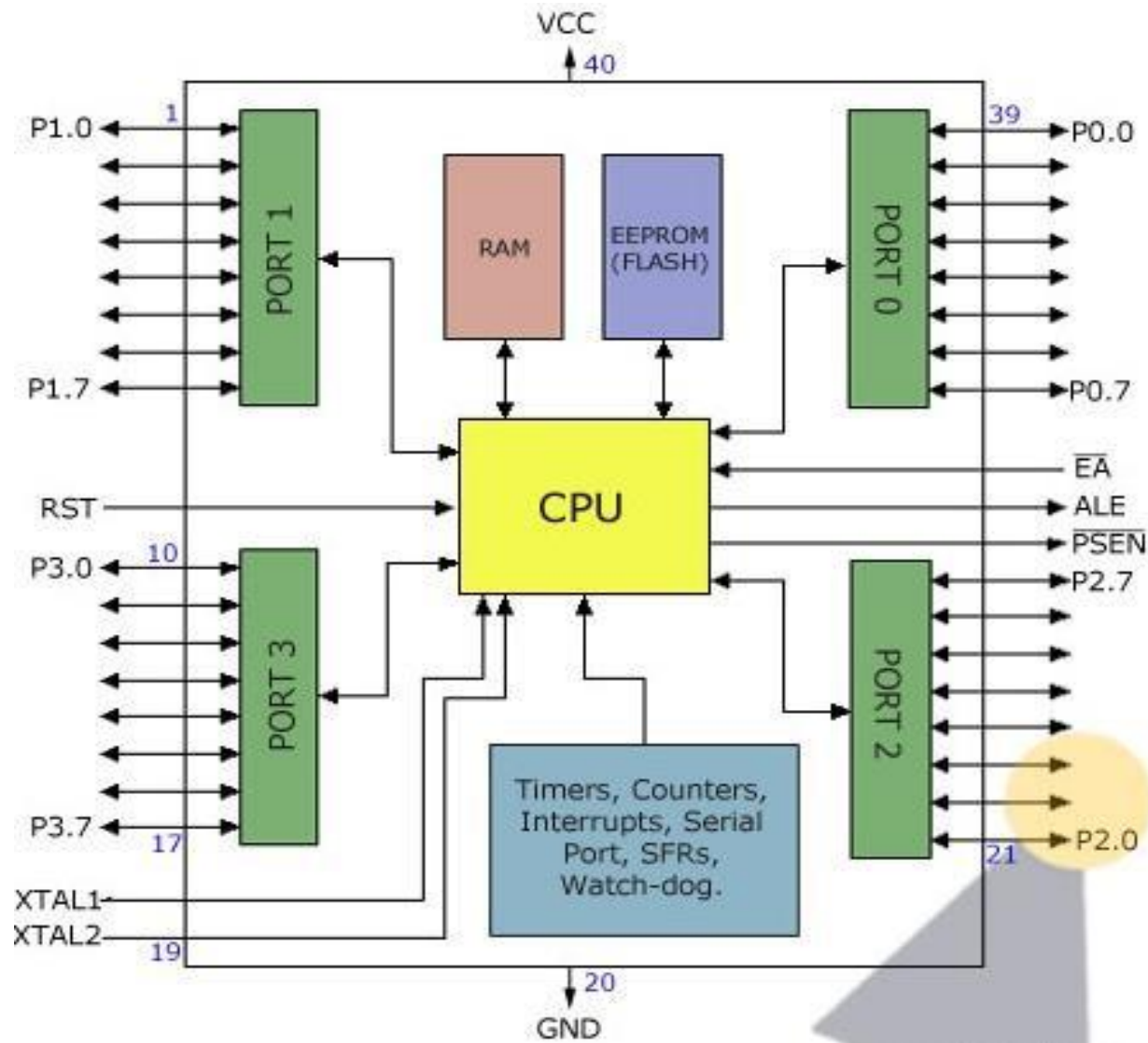
- 8051 family members (e.g., 8751, 89C51, 89C52, DS89C4xO) come in different packages, such as
 - DIP (dual in-line package),
 - QFP (quad flat package), and
 - LLC (leadless chip carrier).
- Have 40 pins that are dedicated to various functions such as I/O, RD, WR, address, data, and interrupts.
- Some companies provide a 20-pin version of the 8051 with a reduced number of I/O ports for less demanding applications.

- In 40 pin package, a total of 32 pins are set aside for the four ports PO, P1, P2, and P3, where each port takes 8 pins.
- The rest of the pins are designated as Vcc, GND, XTAL1, XTAL2, RST, EA, PSEN, and ALE.
- Of these pins, six (Vcc, GND, XTAL1, XTAL2, RST, and EA) are used by all members of the 8051 and 8031 families.
- In other words, they must be connected in order for the system to work, regardless of whether the microcontroller is of the 8051 or 8031 family.
- The other two pins, PSEN and ALE, are used mainly in 8031- based systems.

8051 Pin Diagram



8051 Pin Diagram



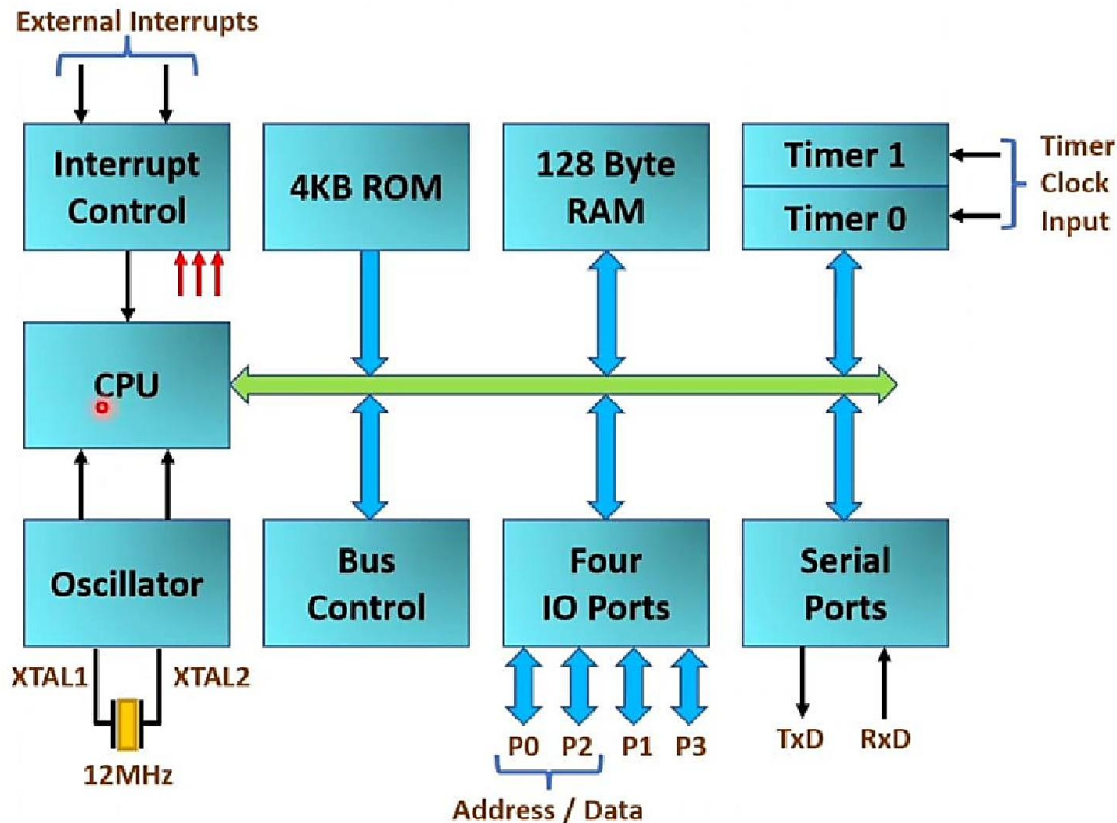
8051 Pin Diagram

- Pins 1 to 8 – These pins are known as Port 1. It is internally pulled up, bi-directional I/O port.
- Pin 9 – It is a RESET pin, which is used to reset the microcontroller to its initial values.
- Pins 10 to 17 – These pins are known as Port 3. This port serves some functions like interrupts, timer input, control signals, serial communication signals RXD and TXD, etc.
- Pins 18 & 19 – These pins are used for interfacing an external crystal to get the system clock.
- Pin 20 – This pin provides the power supply to the circuit.
- Pins 21 to 28 – These pins are known as Port 2. It serves as I/O port. Higher order address bus signals are also multiplexed using this port.

8051 Pin Diagram

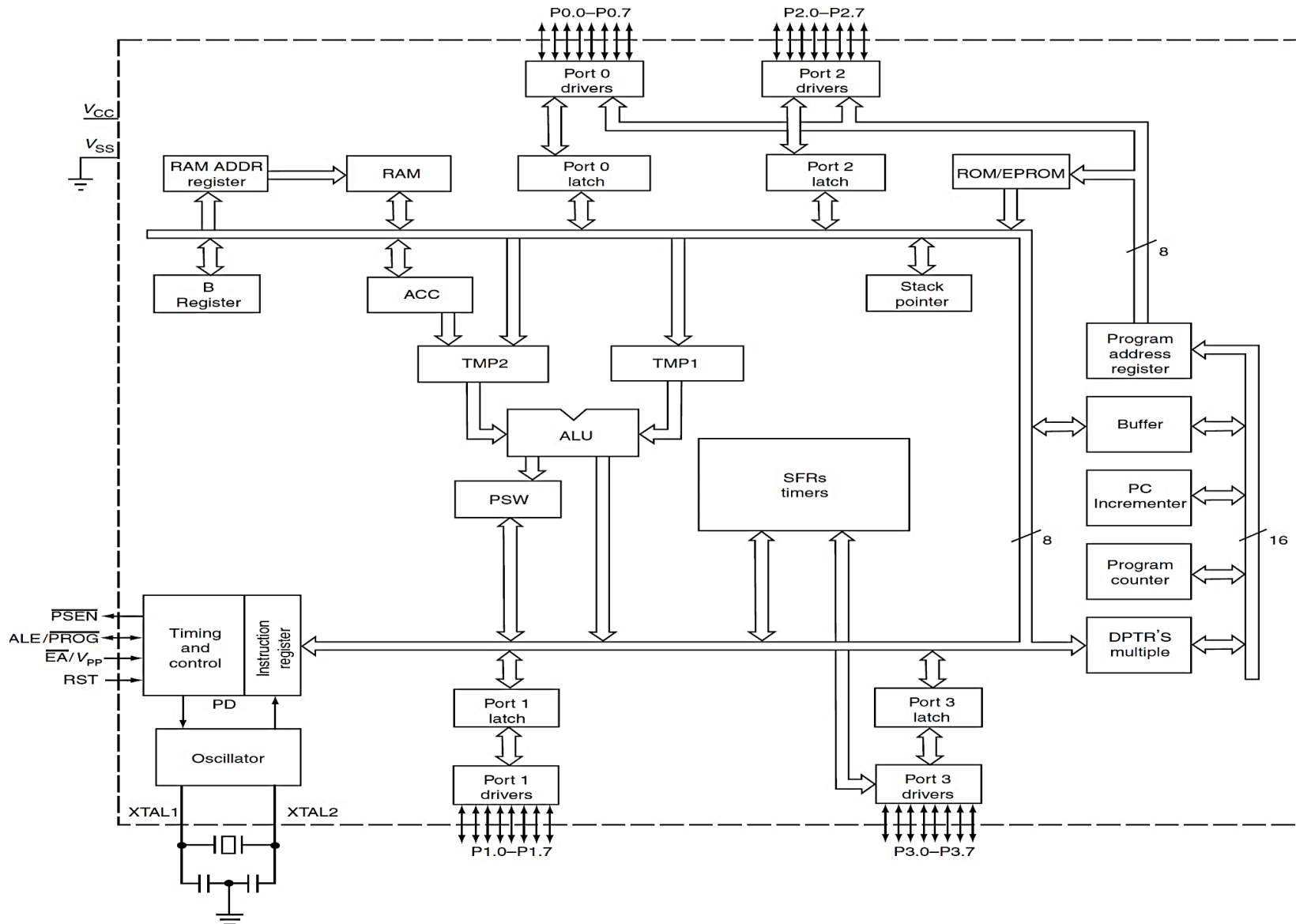
- Pin 29 – This is PSEN pin which stands for Program Store Enable. It is used to read a signal from the external program memory.
- Pin 30 – This is ALE pin which stands for Address Latch Enable. It is used to demultiplex the address-data signal of port.
- Pin 31 – This is EA pin which stands for External Access input. It is used to enable/disable the external memory interfacing.
- Pins 32 to 39 – These pins are known as Port 0. It serves as I/O port. Lower order address and data bus signals are multiplexed using this port.
- Pin 40 – This pin is used to provide power supply to the circuit.

Block diagram of 8051 μ C



- ✓ Harvard Architecture has Separate Memory for Program and Data.
- ✓ Program is stored in ROM and Data is stored in RAM
- ✓ Add Range is 0000H to 0FFFH
- ✓ P0 : AD0-AD7
- ✓ P2 : AD8-AD15

8051 Architecture



CPU (Central Processing Unit)

- CPU act as a mind of any processing machine. It synchronizes and manages all processes that are carried out in microcontroller.
- User has no power to control the functioning of CPU.
- It interprets the program stored in ROM and draws data from storage and performs the projected duty.
- CPU manage the different types of registers available in 8051 microcontroller.
- Its primary elements are an Accumulator (AC), Stack Pointer (SP), Program Counter (PC), Program Status Word (PSW), Data Pointer (DTPR) and few more 8-bit register.

Arithmetic Logic Unit (ALU)

- The arithmetic/logic unit performs the computing functions; it includes the accumulator, temporary register, arithmetic and logic circuits.
- The temporary register is used to hold data during an arithmetic/logic operation. The result is stored in the accumulator register.

CPU (Cont.)

Registers

- The commonly used registers are R0 to R7, A, B, PSW. These are all 8-bit registers. The registers associated with the ALU are A, B and PSW. In most operations, ALU performs specified operation on registers 'A' and 'B' and the result is stored back in 'A'. Hence Register 'A' is often called accumulator.

Control unit

- System Timing, refers to the entire timing unit of the microcontroller. The primary source of timing for 8051 is external crystal. The system timing unit is mainly responsible for providing timing to critical events like instruction fetch, decode and execute operations.

Memory Control

- It is an internal unit of the microcontroller, responsible for accessing data from RAM and ROM.

INTERRUPT

- Interrupt is a subroutine call that interrupts the microcontroller main operations or work and causes it to execute any other program, which is more important at that time of operation.
- The feature of Interrupt is very useful as it helps in case of emergency operations.
- An Interrupts gives us a mechanism to put on hold the ongoing operations, execute a subroutine and then again resumes to another type of operations.
- 8051 can be configured in such a way that it temporarily terminates or pause the main program at the occurrence of interrupts.
- When a subroutine is completed, the execution of main program starts.
- Generally, **five** interrupt sources are there in 8051 Microcontroller.

INTERRUPT CONTROL

- There are five interrupt sources in 8051 Microcontroller and interrupt control section control all these interrupts.
- **Two external interrupts (INT0 & INT1), two timer (TF0 & TF1) interrupts and one serial port (RI / TI) interrupt.**

RAM & ROM

- Microcontroller 8051 has 4K of Code Memory or Program memory i.e. it has **4KB ROM and 128 bytes RAM**.
- The memory which is used to store the program of Microcontroller, is known as code memory or Program memory. It is known as 'ROM' (Read Only Memory).
- Microcontroller also requires a memory to store data or operands temporarily. This memory is known as Data Memory and 'RAM'(Random Access Memory) is used for this purpose.

Data Memory

- Data Memory is often called as **RAM (Random Access Memory)**, which is a type of volatile memory. Program written in to it will not be retained when the power is down or the system is reset.
- Data Memory is used to store temporary data, intermediate results. I

Program Memory

- It is usually implemented as **Read Only Memory or ROM**,
- Program written in to it will be retained even when the power is down or the system is reset.
- The instructions are stored in the Program Memory.

BUS CONTROL

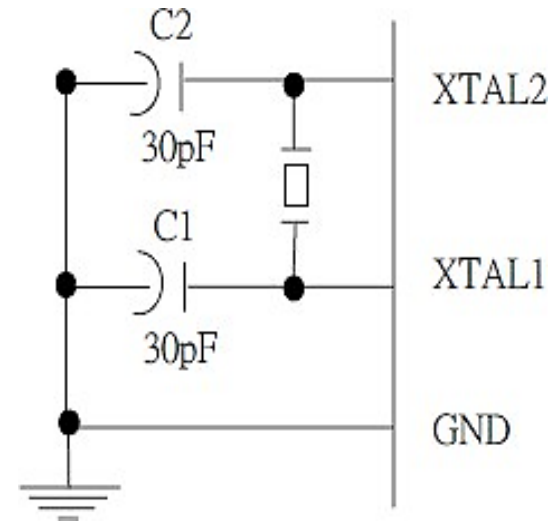
- Bus control section of 8051 is responsible for controlling the operation of address and data bus.
- **Bus: Bus is a collection of wires which work as a communication channel or medium for transfer of Data.**
- Address Bus: Microcontroller 8051 has a **16-bit address bus**. It used to address memory locations.
- Data Bus: Microcontroller 8051 has **8 bits data bus**. It is used to carry data.

CRYSTAL OSCILLATOR

- Crystal Oscillator: Since Microcontroller is a digital circuit device, therefore it **requires clock for its operation**.
- For this purpose, Microcontroller 8051 has oscillator circuitry section which works as a clock source for Central Processing Unit.
- As the output pulses of oscillator are stable therefore it enables synchronized work of all parts of 8051 Microcontroller.

XTAL1 and XTAL2

- The 8051 has an on-chip oscillator but requires an external crystal to run it.
 - A quartz crystal oscillator is connected to inputs XTAL1 (pin19) and XTAL2 (pin18)
 - The quartz crystal oscillator also needs two capacitors of 30 pF value
 - The original 8051 operates at 11.059 MHz



TIMERS/COUNTERS

- Timers/Counters: 8051 Microcontroller has two 16-bit timers and counters: **Timer 0 and Timer 1**.
- They can be used either as timers to generate a time delay or as counters to count events happening outside the microcontroller.
- Since the 8051 has an 8-bit architecture, each 16-bit is accessed as two separate registers of low byte and high byte

SERIAL PORT

- 8051 contains one Serial port or UART (Universal Asynchronous Receiver Transmitter)
- The serial port is full-duplex so, it can transmit and receive simultaneously
- Two port pins are used to provide the serial interface P3.0 is the receive pin (RXD) P3.1 is the transmit pin (TXD)
- This serial port that can be programmed to operate in one of four different modes and at a range of frequencies.

I/O PORTS

Input/Output Port

I/O Ports provide the microcontroller, a physical connection to the outside world.

Input Ports provide a gateway for passing on the data from the outside world with the help of sensors.

Output Ports allow microcontroller to control external devices (like motors and LEDs).

I/O PORTS

- I/O Ports: To connect any external devices or peripherals we require I/O interfacing ports in the microcontroller.
- All 8051 microcontrollers have 4 I/O ports each comprising 8 bits which can be configured as input (1) or an output (0), depends on its logic state.
- Accordingly, in total of 32 input/output pins enabling the microcontroller to be connected to peripheral devices are available for use.

I/O Ports & their functions

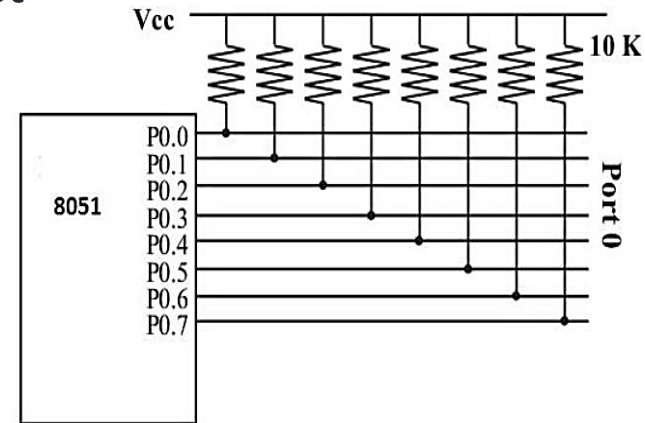
- There are four ports: P0, P1, P2 and P3.
- Each port are bi-directional and has 8 pins.
- Upon RESET, all the ports are configured as inputs, or ready to be used as input ports.
- If programmer writes '0' to it, then port works as output. If programmer needs to configure as input, a '1' must be sent to port.

Port-0

- It has 8 pins (32 to 39).
- It can be used for input or output.
- Unlike P1, P2, and P3 ports, we normally connect P0 to 10K-ohm pull-up resistors to use it as an input or output port being an open drain.

What is pull-up register?

- Ans: It is a fixed value register. Pull-up resistors are resistors which are used to ensure that a wire is pulled to a high logical level in the absence of an input signal.



Port-1

- It is an 8-bit port (pin 1 through 8) and can be used either as input or output. It doesn't require pull-up resistors because they are already connected internally.

Port-2

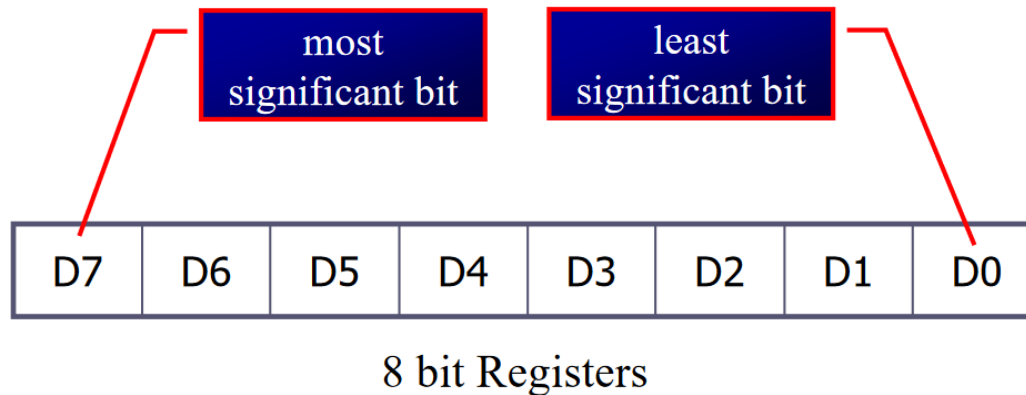
- Port 2 occupies a total of 8 pins (pins 21 through 28) and can be used for both input and output operations.
- Just as P1 (Port 1), P2 also doesn't require external Pull-up resistors because they are already connected internally.
- It must be used along with P0 to provide the 16-bit address for the external memory.

Port-3

- It is also of 8 bits and can be used as Input/Output. This port provides some extremely important signals.
- P3.0 and P3.1 are RxD (Receiver) and TxD (Transmitter) respectively and are collectively used for Serial Communication.
- P3.2 and P3.3 pins are used for external interrupts.
- P3.4 and P3.5 are used for timers T0 and T1 respectively.
- P3.6 and P3.7 are Write (WR) and Read (RD) pins.
- These are active low pins, means they will be active when 0 is given to them and these are used to provide Read and Write operations to External ROM in 8031 based systems.

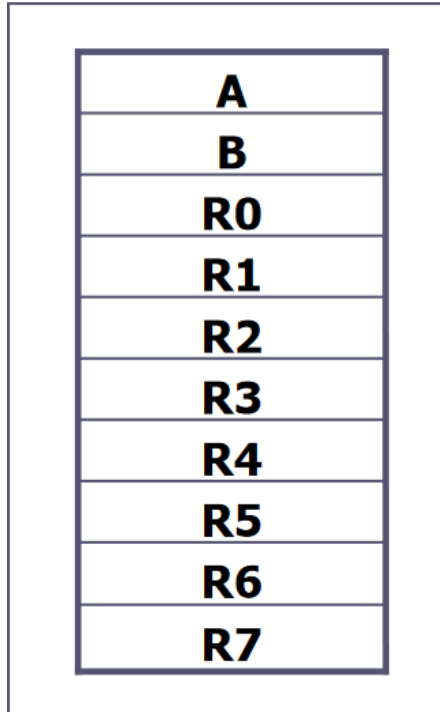
Register are used to store information temporarily, while the information could be

1. A byte of data to be processed, or
2. An address pointing to the data to be fetched

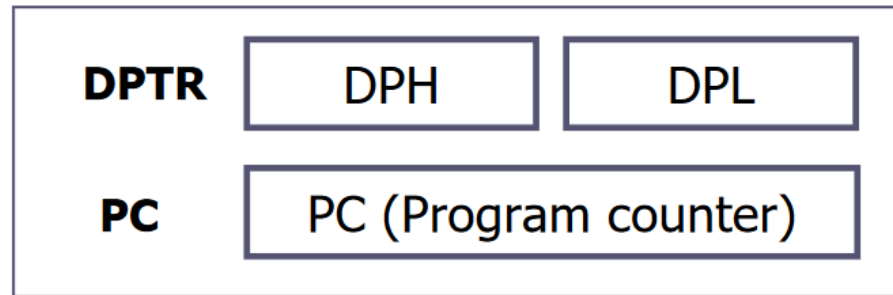


The most widely used registers

Some 8051 8-bit Registers



Some 8051 16-bit Registers



Registers

- The 8051 microcontroller has a variety of registers that play a critical role in its operation.
- These registers are used to store data temporarily, perform arithmetic and logical operations, control the program flow, and manage peripherals.

1. Accumulator (A)

- Type: 8-bit
- Central register used for arithmetic, logical operations, and data transfer.
- Often referred to as ACC.
- Common Instructions: ADD, SUBB, MUL, DIV, ANL, ORL, MOV.

2. B Register

- Type: 8-bit
- Used in conjunction with the Accumulator for multiplication (MUL AB) and division (DIV AB) instructions.
- Can also serve as a temporary storage register for general-purpose operations.

3. Data Pointer (DPTR)

- Type: 16-bit (split into DPL and DPH)
- Used for indirect addressing, especially **for accessing external memory** or program memory.
- Consists of two 8-bit registers:
 - DPH: Data Pointer High
 - DPL: Data Pointer Low
- In any computer system, data is kept at a particular address, and the DPTR is responsible for storing the address.
- Common Instructions: `MOV DPTR, #data16;` `MOVB A, @DPTR;` `MOVC A, @A+DPTR.`

4. Program Counter (PC)

- Type: 16-bit
- Holds the address of the next instruction to be executed.
- Automatically increments after fetching an instruction.
- Used for program flow control with jumps, calls, and interrupts.
- Range: 0000H to FFFFH (64 KB of code memory).

5. Stack Pointer (SP)

- Type: 8-bit
- Points to the top of the stack in internal RAM.
- Defaults to 07H at reset, meaning the stack starts at address 08H.
- Increments automatically during a PUSH and decrements during a POP.

6. Special Function Registers (SFRs)

- Type: 8-bit (accessible only in address range 80H to FFH)
- Used to control and monitor various functions of the microcontroller.
- Some important SFRs:
- P0, P1, P2, P3: Port registers for I/O operations.
- TCON, TMOD: Timer control and mode registers.
- IE, IP: Interrupt enable and priority registers.
- SCON, SBUF: Serial communication registers.
- PCON: Power control register.

7. General-Purpose Registers (R0 – R7)

- Type: 8-bit
- Used for temporary storage and data manipulation during execution.
- Organized into four banks, with each bank containing eight registers (R0–R7).
- Bank selection is controlled by the PSW (Program Status Word) register.
- Default bank: Bank 0.

8. Program Status Word (PSW)

- Type: 8-bit
- Holds flags and control bits for arithmetic operations and bank selection.
- Important bits:
 - CY: Carry flag
 - AC: Auxiliary carry flag
 - OV: Overflow flag
 - P: Parity flag
 - RS1, RS0: Register bank select bits.

9. Timer Registers (TH0, TL0, TH1, TL1)

- Type: 8-bit
- Used for timer/counter operations.
- High (THx) and low (TLx) bytes represent the 16-bit timer value.
- Controlled by TMOD and TCON registers.
- Timer Control (TCON)– Controls the two timers on 8051.
- Timer Mode (TMOD)– This SFR is used to select the mode of the timer.

10. Serial Data Registers (SCON, SBUF)

- SCON: Serial Control Register
- Configures the serial port for communication (mode, baud rate, etc.).
- SBUF: Serial Buffer Register
- Temporary storage for data during serial communication (transmit and receive).

11. Interrupt Registers (IE, IP)

- IE: Interrupt Enable Register
- Enables or disables specific interrupts.
- IP: Interrupt Priority Register
- Sets priority levels for interrupts.

12. Power Control (PCON)

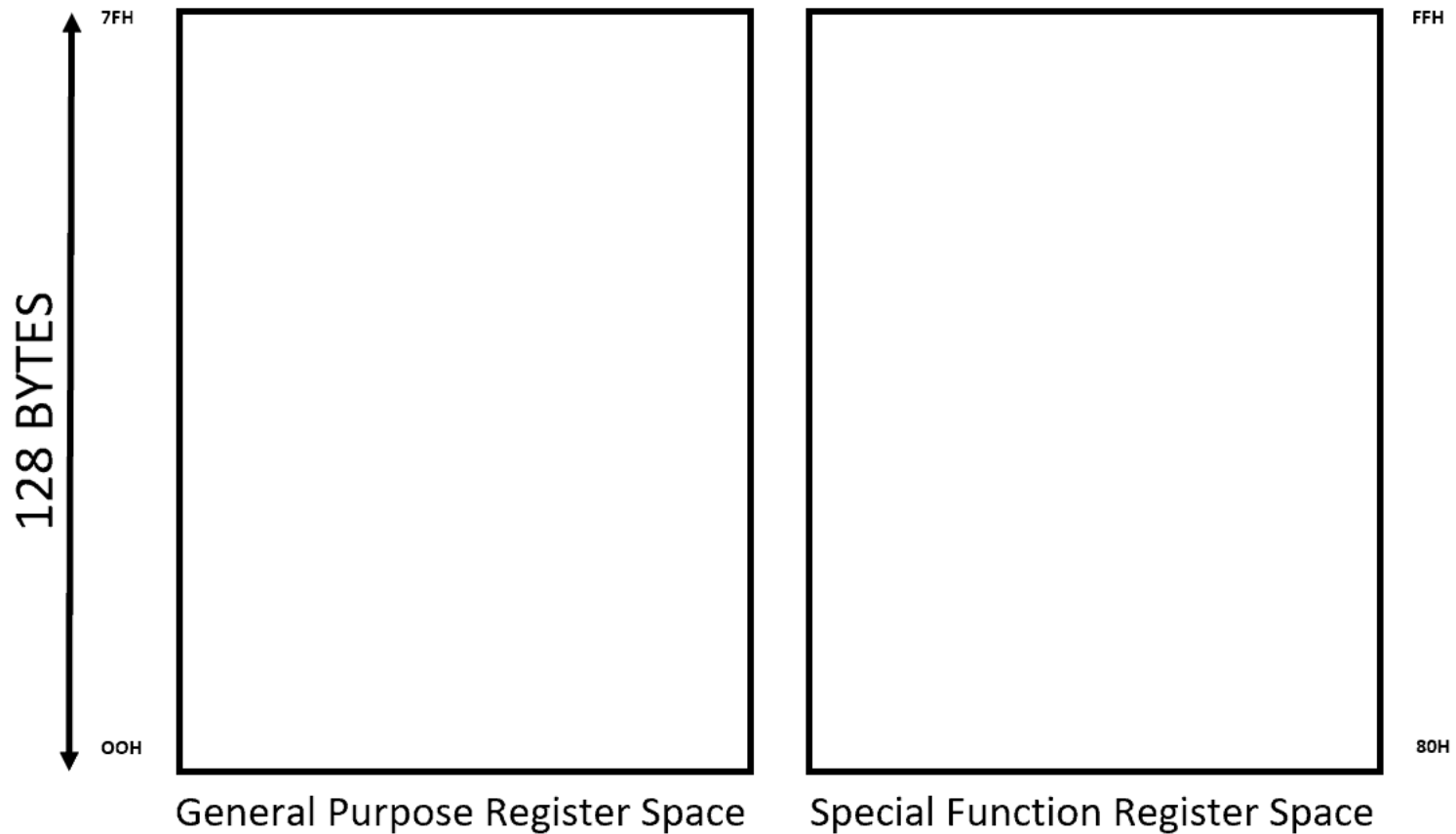
- Selects various power saving modes of 8051.

13. PC Incrementer

- The program code is sequentially burned into the flash.
- So, the PC Incrementer is responsible for increasing the address in the program counter register by one.
- This makes it a point to the next instruction, which needs to be executed..

Special Function Registers

- The general-purpose register space stores all the data that generates during the execution of a program.
- In contrast, the special function registers are responsible for managing the various operations occurring in the microcontroller.
- The 8051 microcontroller has 21 special function registers (SFRs).
- SFRs are located in the upper area from address 0x80 to 0xFF.
- SFRs are used to control and monitor the operation of the microcontroller.
- They contain registers for peripherals, timers, counters, serial parts, and interrupts.
- SFRs can be accessed as bits, bytes, or words.



RAM memory partition in 8051

Special Function Registers

80	P0	SP	DPL	DPH				PCON	87
88	TCON	TMOD	TL0	TL1	TH0	TH1			8F
90	P1								97
98	SCON	SBUF							9F
A0	P2								A7
A8	IE								AF
B0	P3								B7
B8	IP								B9
C0									C7
C8									CF
D0	PSW								D7
D8									DF
E0	ACC								E7
E8									EF
F0	B								F7
F8									FF



Blue background are I/O port SFRs
 Yellow background are control SFRs
 Green background are other SFRs

8051 Special Function Register (SFR)

Symbol	Name	Address
ACC*	Accumulator	0E0H
B*	B register	0F0H
PSW*	Program status word	0D0H
SP	Stack pointer	81H
DPTR	Data pointer 2 bytes	
DPL	Low byte	82H
DPH	High byte	83H
P0*	Port 0	80H
P1*	Port 1	90H
P2*	Port 2	0A0H
P3*	Port 3	0B0H
IP*	Interrupt priority control	0B8H
IE*	Interrupt enable control	0A8H
TMOD	Timer/counter mode control	89H
TCON*	Timer/counter control	88H

T2CON*	Timer/counter 2 control	0C8H
T2MOD	Timer/counter mode control	0C9H
TH0	Timer/counter 0 high byte	8CH
TL0	Timer/counter 0 low byte	8AH
TH1	Timer/counter 1 high byte	8DH
TL1	Timer/counter 1 low byte	8BH
TH2	Timer/counter 2 high byte	0CDH
TL2	Timer/counter 2 low byte	0CCH
RCAP2H	T/C 2 capture register high byte	0CBH
RCAP2L	T/C 2 capture register low byte	0CAH
SCON*	Serial control	98H
SBUF	Serial data buffer	99H
PCON	Power control	87H

* Bit-addressable

General-Purpose Registers in 8051

Register Banks:

- 8051 has 32 general-purpose registers, divided into four banks:
 - Bank 0: R0–R7
 - Bank 1: R0–R7
 - Bank 2: R0–R7
 - Bank 3: R0–R7
- Each register is 8 bits wide.
- By default, the 8051 uses Bank 0.
- Location in RAM:
 - These registers are stored in the lower 32 bytes of the internal RAM (00H to 1FH).
 - Each bank occupies 8 bytes:
 - Bank 0: Addresses 00H–07H
 - Bank 1: Addresses 08H–0FH
 - Bank 2: Addresses 10H–17H
 - Bank 3: Addresses 18H–1FH

Program Status Word (PSW) / Flag Register



❖ P – Parity Flag

- ☐ P = 1, Odd Parity {Odd Number of 1's in result}
- ☐ P = 0, Even Parity {Even Number of 1's in result}

❖ OVR – Overflow Flag

- ☐ OVR = 1, Signed overflow
- ☐ OVR = 0, No Signed overflow
- ☐ It happens when result goes beyond 127 to -128.
- ☐ After overflow, sign of result {MSB} becomes wrong.

❖ RS – Register Bank Select

- ☐ RS = 00, Register Bank 0, {Default}
- ☐ RS = 01, Register Bank 1
- ☐ RS = 10, Register Bank 2
- ☐ RS = 11, Register Bank 3
- ☐ By CLR and SETB instructions we can select register bank.

Example

CLR PSW.4

SETB PSW.3 ;Here RS = 01 means bank 1 is selected

Example:
ADD A,R1

	11	1	1
A = CCH	1100	1100	
R1 = E6H	1110	0110	
A = B2H	1011	0010	

❖ F0 – User Defined Flag

- ☐ Set by user using SETB PSW.5
- ☐ Clear by user using CLR PSW.5

❖ AC – Axillary Carry Flag

- ☐ AC = 1, Nibble to Nibble Carry
- ☐ AC = 0, No Nibble to Nibble Carry

❖ CY – Carry Flag

- ☐ CY = 1, Result has Carry.
- ☐ CY = 0, Result has no Carry.

$$\begin{array}{r}
 \\
 11 11 \\
 A = CCH 1100 \\
 R1 = E6H 0110 \\
 \hline
 A = B2H 0010
 \end{array}$$

Bit	7 (CY)	6 (AC)	5 (F0)	4 (RS1)	3 (RS0)	2 (OV)	1 (Unused)	0 (P)
Value	1	1	0	0	0	0	0	0

This gives us the binary PSW value:

$$PSW = 11000000_2 = C0H$$

Show the status of the CY, AC, and P flags after the addition of 38H and 2FH in the following instructions.

MOV A,#38H

ADD A,#2FH

Solution:

38	00111000
<u>+ 2F</u>	<u>00101111</u>
67	01100111

;after the addition A=67H, CY=0

CY = 0 since there is no carry beyond the D7 bit.

AC = 1 since there is a carry from the D3 to the D4 bit.

P = 1 since the accumulator has an odd number of 1s (it has five 1s).

Show the status of the CY, AC, and P flags after the addition of 9CH and 64H in the following instructions.

MOV A,#9CH
ADD A,#64H

Solution:

9C	10011100
<u>+ 64</u>	<u>01100100</u>
100	00000000

;after addition A=00 and CY=1

CY = 1 since there is a carry beyond the D7 bit.

AC = 1 since there is a carry from the D3 to the D4 bit.

P = 0 since the accumulator has an even number of 1s (it has zero 1s).

Show the status of the CY, AC, and P flags after the addition of 88H and 93H in the following instructions.

MOV A,#88H

ADD A,#93H

Solution:

88	10001000
<u>+ 93</u>	<u>10010011</u>
11B	00011011

;after the addition A=1BH,CY=1

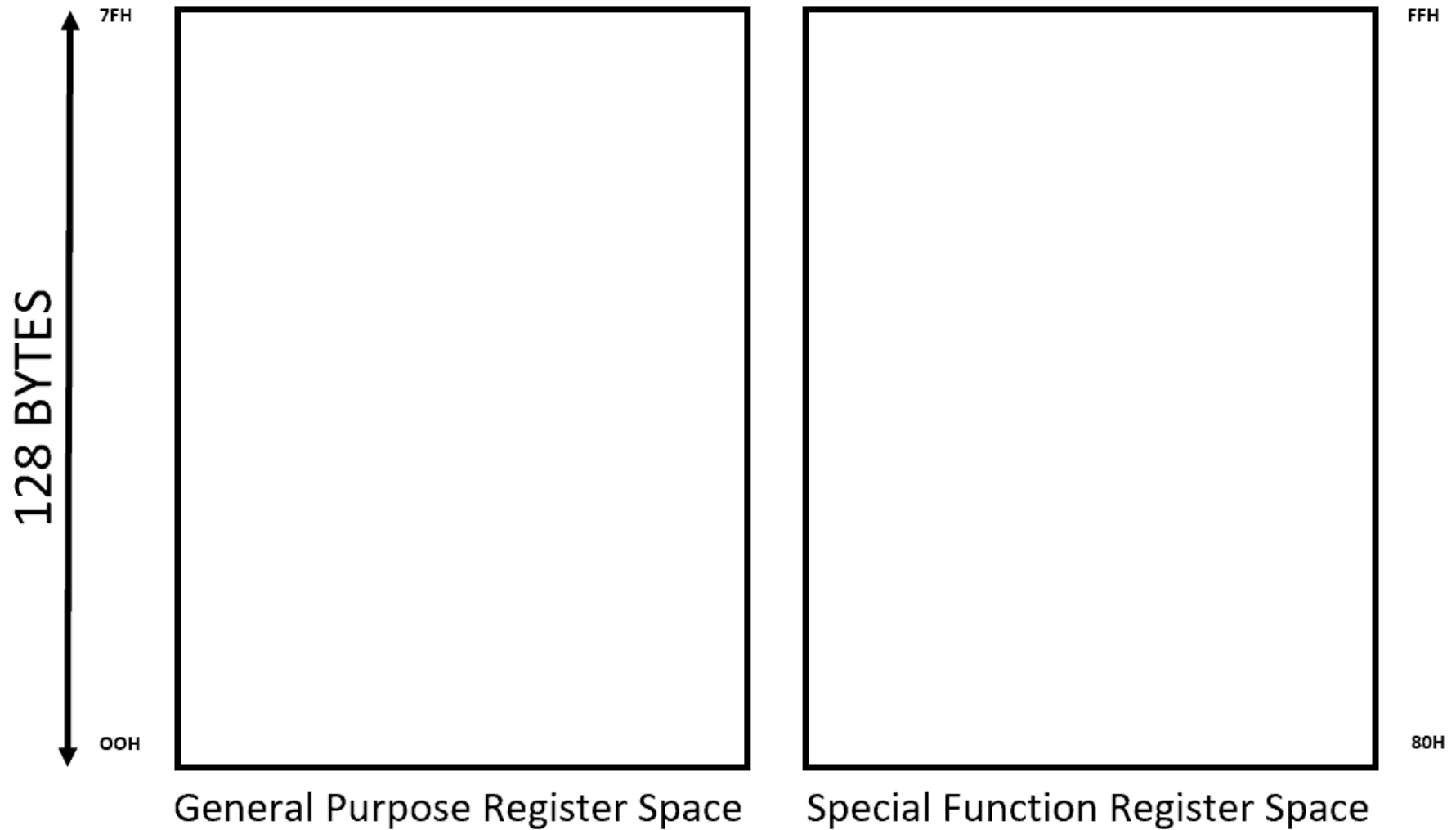
CY = 1 since there is a carry beyond the D7 bit.

AC = 0 since there is no carry from the D3 to the D4 bit.

P = 0 since the accumulator has an even number of 1s (it has four 1s).

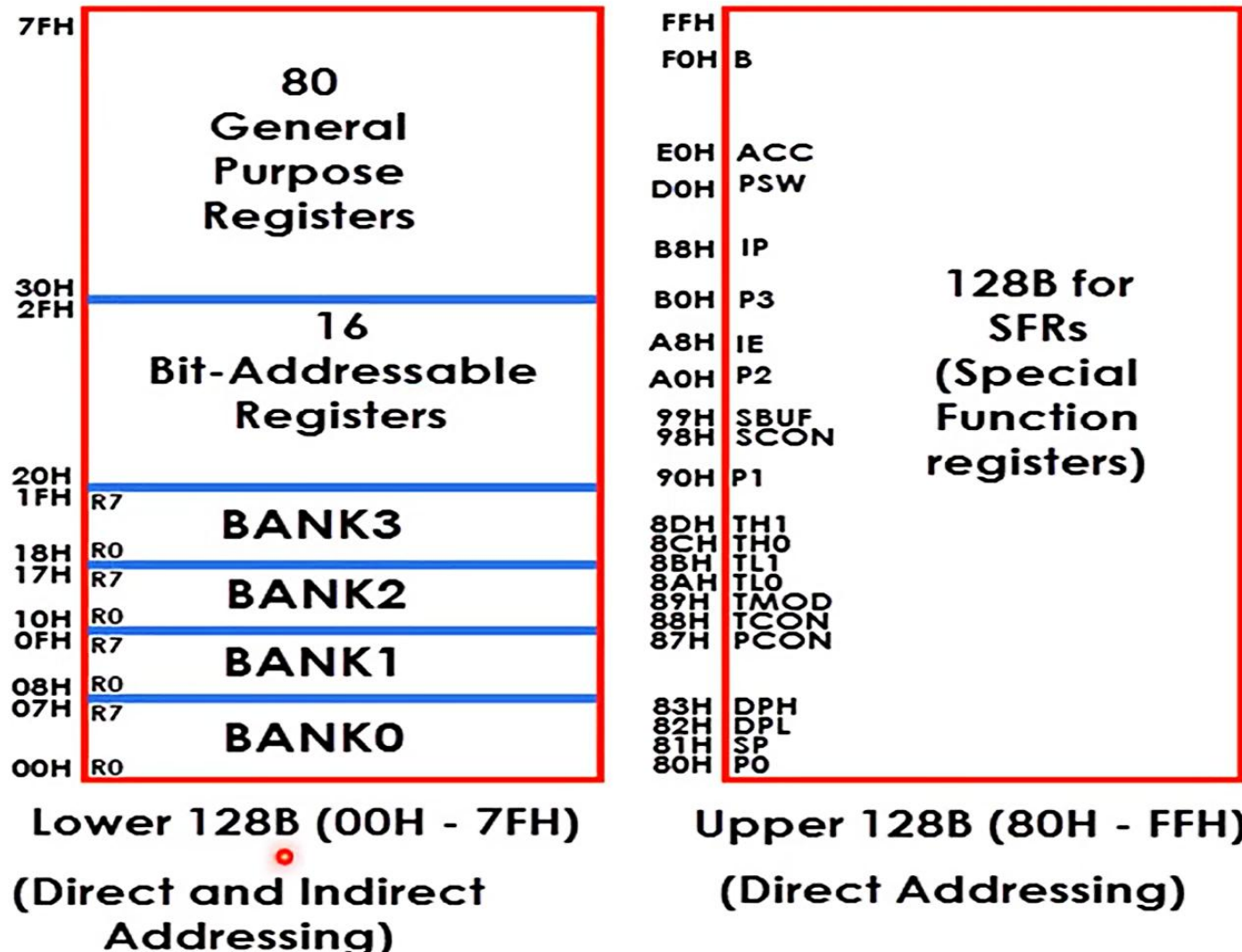
RAM-ROM Organization of 8051

RAM Memory Partition in 8051



RAM memory partition in 8051

RAM Structure



Internal RAM Structure

❖ 8051 Microcontroller Register Bank in RAM

- ❑ 8051 has four register banks. Each register bank has Eight registers R0-R7.
- ❑ Selection of register bank can be done by two bits of PSW register, by PSW.3 and PSW.4 we can select any register bank.

CLR PSW.4

SETB PSW.3 ;Here RS = 01 means bank 1 is selected

- ❑ RAM address 00H to 1FH holds four register banks.

MOV A,R0 ;Copy R0 into A

MOV A,08H ;Copy R0 of register bank 1 into A

❖ 8051 Microcontroller Bit Addressable Area in RAM

- ❑ 16 Byte of RAM from 20H to 2FH holds bit addressable area in internal RAM of 8051 microcontroller.
- ❑ $16 \times 8 = 128$ Addressable bits in these area.
- ❑ These bits are addressed as per 00H to 7FH, in total 128 bits.
- ❑ These locations can have bit as well byte wise operations.

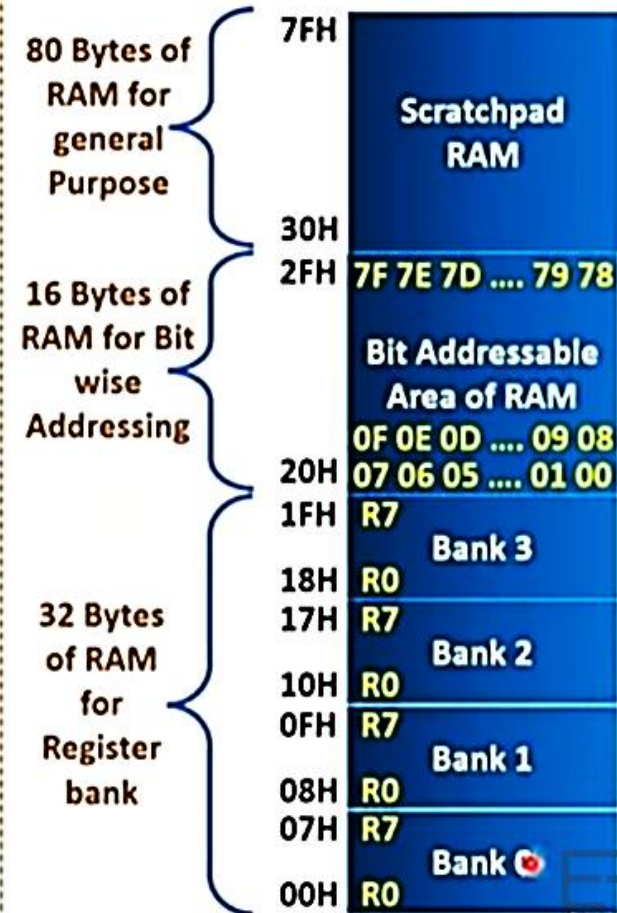
SETB 7FH ;Set MSB of 2FH RAM location

CLR 08H ;Clear LSB of 21H RAM location

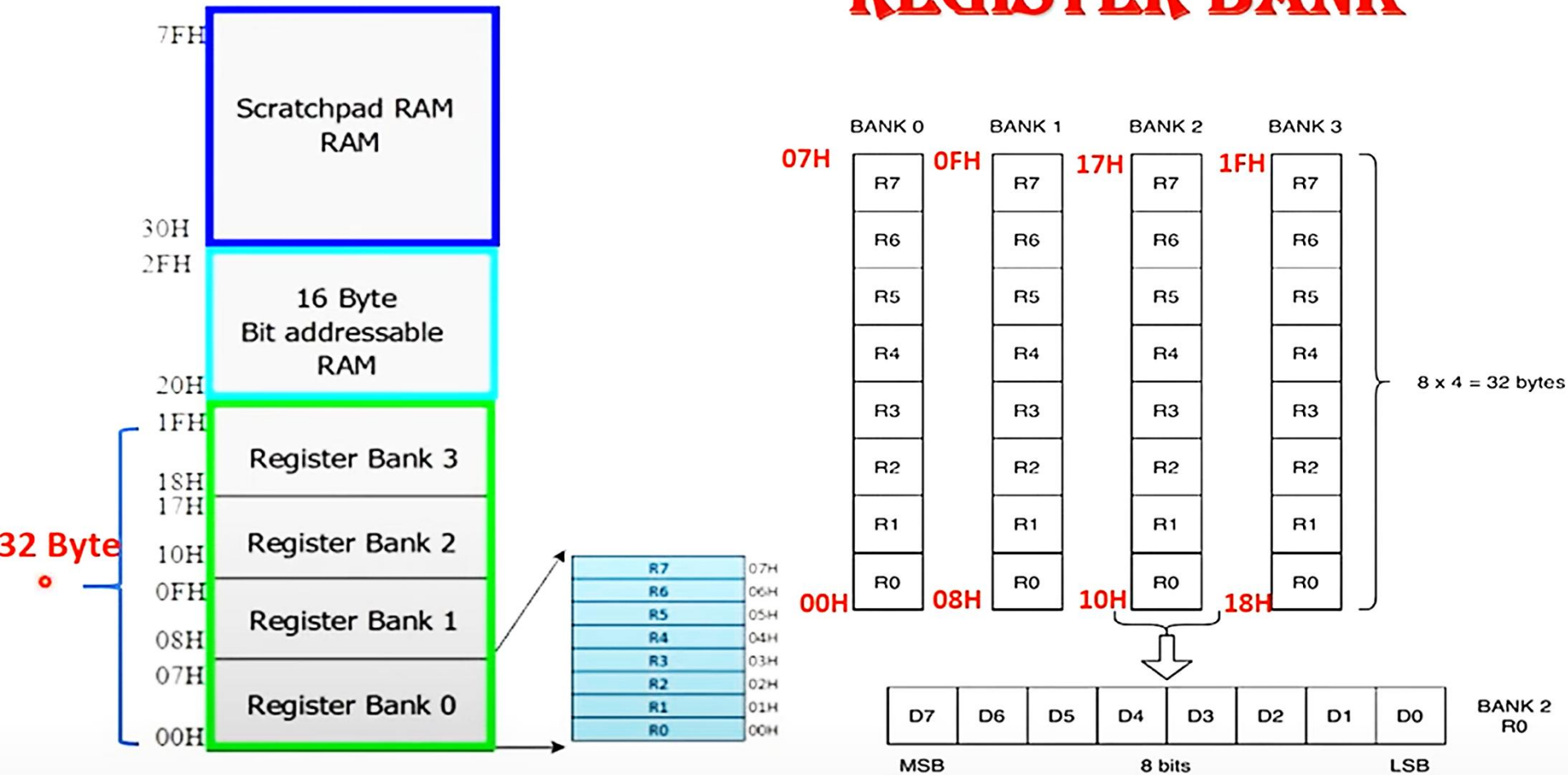
MOV 20H, #FFH ;Set all bits 20H RAM locations

❖ 8051 Microcontroller General Purpose Area in RAM {Scratchpad RAM}

- ❑ 80 Byte of RAM from 30H to 7FH holds general purpose area in internal RAM of 8051 microcontroller.
- ❑ It can be used for general purpose operations.

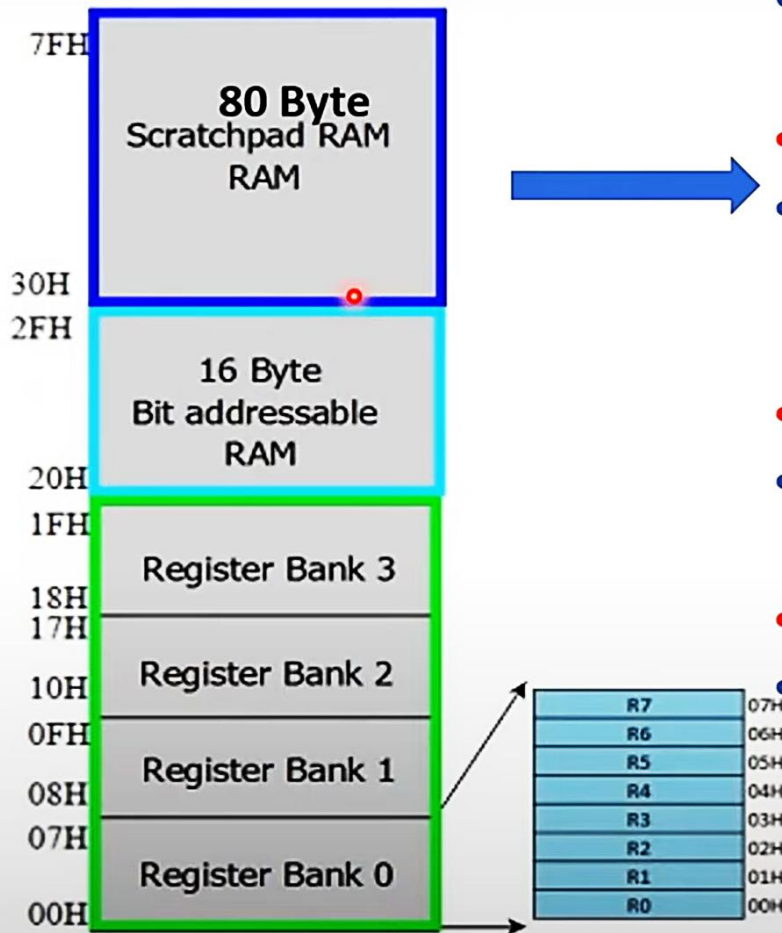


REGISTER BANK



Bit Addressable Registers





- **Scratch pad area** (general purpose area) consists 80 bytes of memory.
- **It's address starting from 30H to 7FH.**
- Whenever stack pointer area has filled, then data will be stored in scratch pad area.
- **This area can be use to store Results.**
- It can be use to store given series of data byte.
- **It can be use to store the addresses.**
- It is also use as a stack memory area. It is also called high speed internal memory area.

ROM Organization of 8051

❖ 8051 Microcontroller Memory

❑ 8051 can have four different memories:

- Internal ROM {4KB}
- Internal RAM {128 byte}
- External ROM {Max 64KB}
- External RAM {Max 64KB}

❖ Architecture of 8051 Microcontroller

- ❑ 8051 follows Harvard Architecture.
- ❑ So, with 8051 Program memory is ROM and Data memory is RAM.

❖ Applications of 8051 Microcontroller

- ❑ Microcontroller is used in many embedded system applications. {Remote control, Microwave Oven, Washing Machine etc.}
- ❑ Once controller is embedded in application, program of controller should be fixed and it should be stored in ROM.
- ❑ Data may change in application like time, temperature and it will be stored in RAM.
- ❑ Fixed data will be stored in ROM as look up table. {ASCII, SSD etc.}

ROM Organization

Only Internal ROM

$\overline{EA} = 1$



Internal & External ROM

$\overline{EA} = 1$

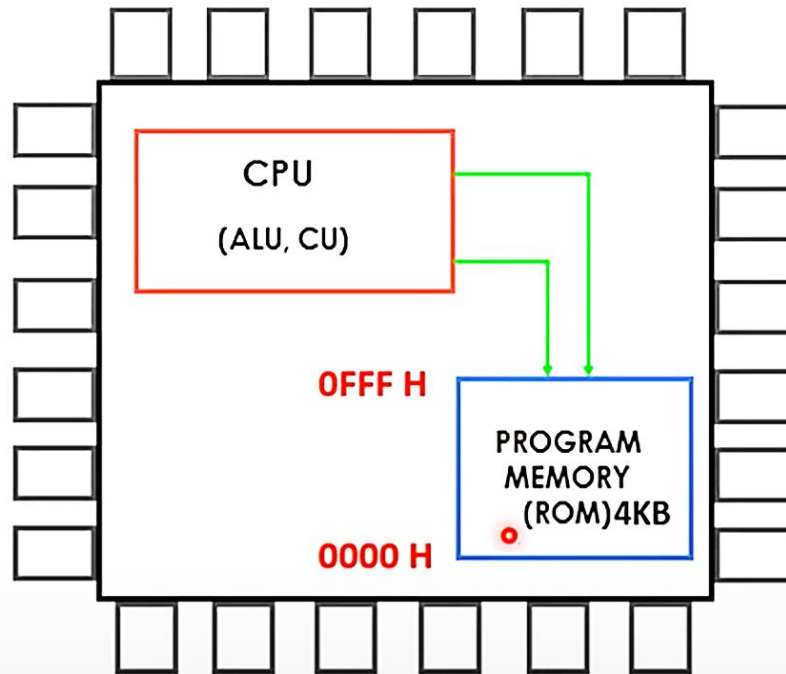


Only External ROM

$\overline{EA} = 0$

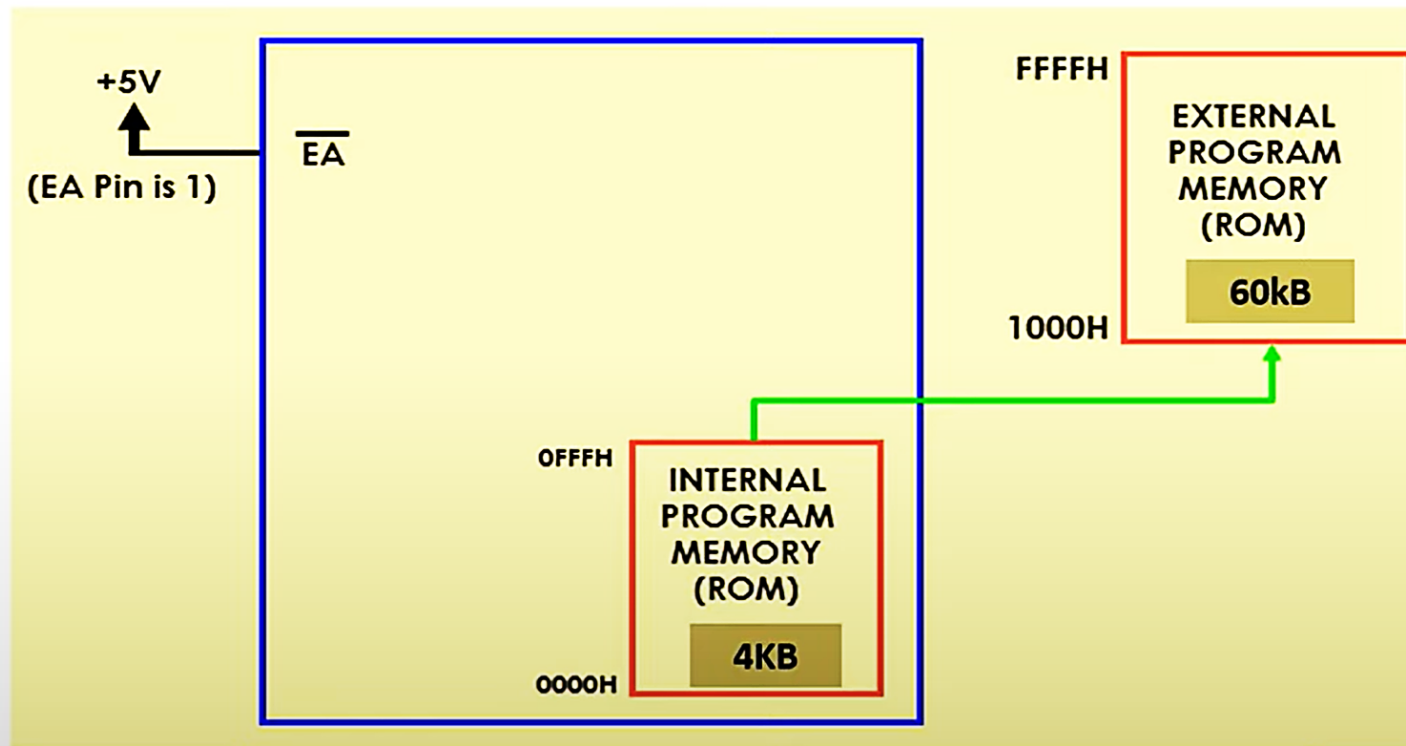


- The code or instructions to be executed are stored in the Program Memory (ROM) in 8051 MC.
- The original 8051 MC by Intel has 4KB of internal ROM.

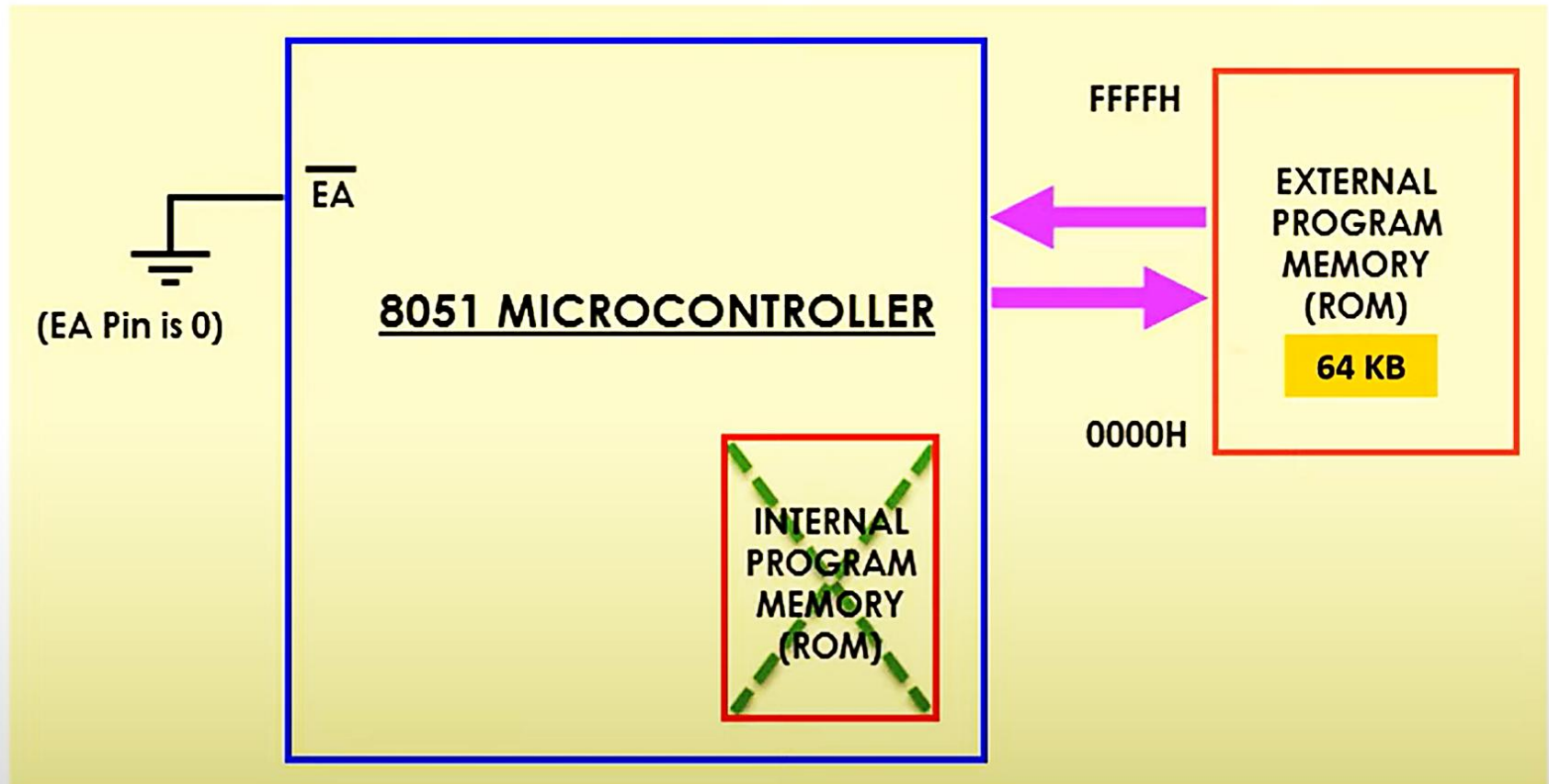


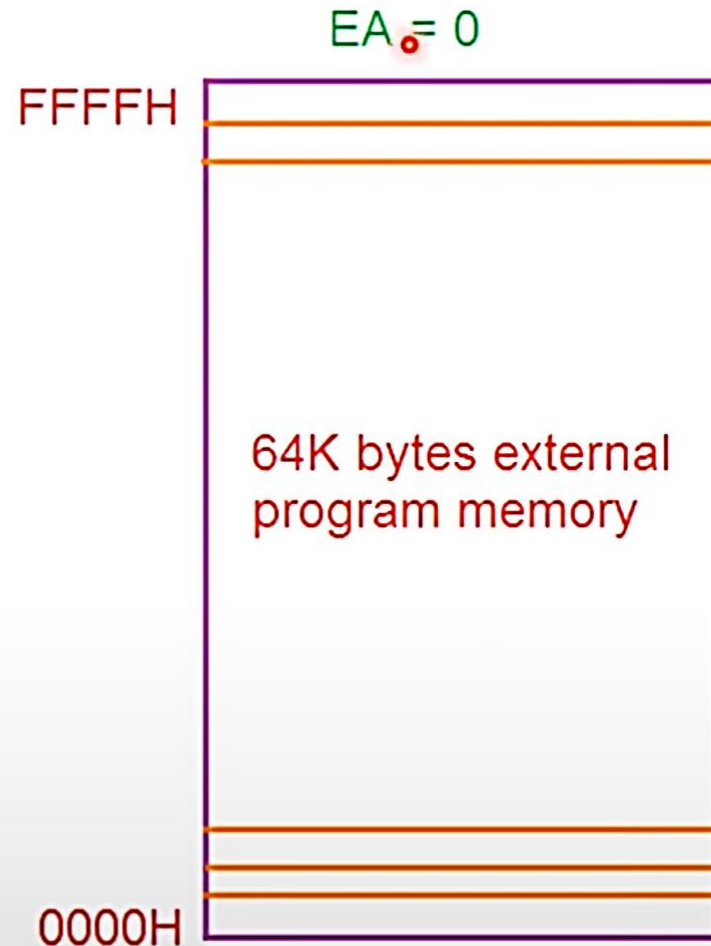
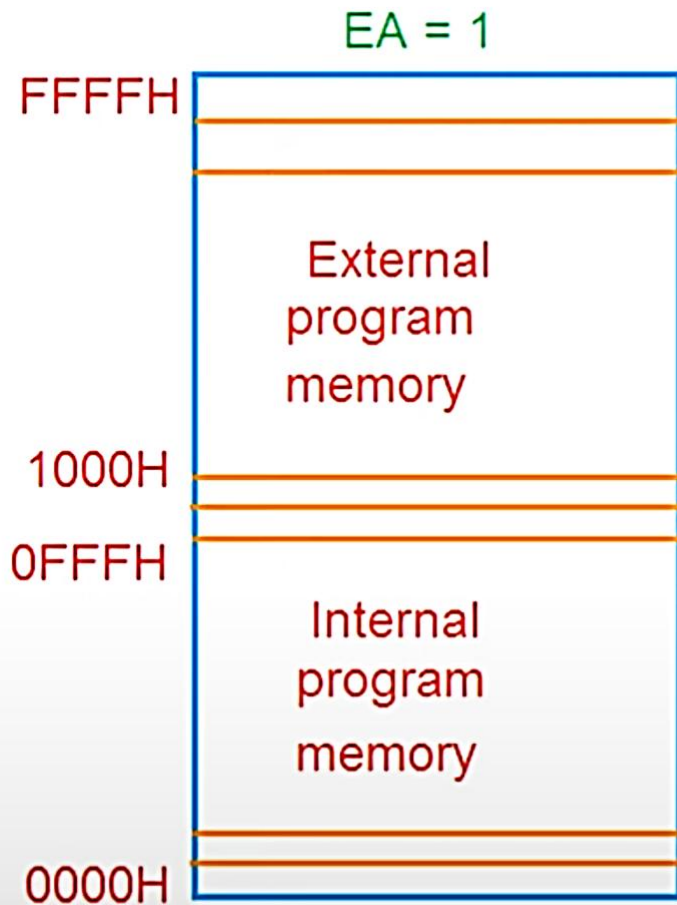
But If we want to extend the program memory , extend upto 64 KB in two way's

1) 4KB (Internal ROM) + 60 KB (External ROM)



2) Use only 64 KB (External ROM)





State the contents of RAM locations after the following program:

MOV R0,#99H ;load R0 with value 99H

MOV R1,#85H ;load R1 with value 85H

MOV R2,#3FH ;load R2 with value 3FH

MOV R7,#63H ;load R7 with value 63H

MOV R5,#12H ;load R5 with value 12H

Solution:

After the execution of the above program we have the following:

RAM location 0 has value 99H

RAM location 1 has value 85H

RAM location 2 has value 3FH

RAM location 7 has value 63H

RAM location 5 has value 12H

Repeat previous Example using RAM addresses instead of register names.

Solution:

This is called direct addressing mode and uses the RAM address location for the destination address.

MOV 00,#99H ;load RAM Address with value 99H

MOV 01,#85H ;load RAM Address with value 85H

MOV 02,#3FH ;load RAM Address with value 3FH

MOV 07,#63H ;load RAM Address with value 63H

MOV 05,#12H ;load RAM Address with value 12H

State the contents of the RAM locations after the following program:

```
SETB PSW.4    ;select bank 2
MOV R0,#99H    ;load R0 with value 99H
MOV R1,#85H    ;load R1 with value 85H
MOV R2,#3FH    ;load R2 with value 3FH
MOV R7,#63H    ;load R7 with value 63H
MOV R5,#12H    ;load R5 with value 12H
```

Solution:

By default, PSW.3=0 and PSW.4=0; therefore, the instruction “SETB PSW.4” sets RS1=1 and RS0=0, thereby selecting register bank 2. Register bank 2 uses RAM locations 10H - 17H. After the execution of the above program we have the following:

RAM location 10H has value 99H

RAM location 11H has value 85H

RAM location 12H has value 3FH

RAM location 17H has value 63H

RAM location 15H has value 12H

State the contents of the RAM locations after the following program:

```
SETB PSW.3    ;select bank ?  
MOV R0,#99H   ;load R0 with value 99H  
MOV R1,#85H   ;load R1 with value 85H  
MOV R2,#3FH   ;load R2 with value 3FH  
MOV R7,#63H   ;load R7 with value 63H  
MOV R5,#12H   ;load R5 with value 12H
```

Solution:

Here, PSW.3=1 and by-default PSW.4=0; therefore, the instruction “SETB PSW.3” sets RS0=1 and RS1=0, thereby selecting register bank 1. Register bank 1 uses RAM locations 08H – 0FH. After the execution of the above program we have the following:

RAM location 08H has value 99H	RAM location 09H has value 85H
RAM location 0AH has value 3FH	RAM location 0FH has value 63H
RAM location 0DH has value 12H	

PUSH and POP Instruction

These two are stack related instructions.

- Stack is a section of a RAM, there are registers inside the CPU to point to it.
- The register used to access the stack is known as the stack pointer register.
- The stack pointer in the 8051 is 8-bits wide, and it can take a value of 00 to FFH.
- When the 8051 is initialized, the SP register contains the value 07H.

[Subscribe Now](#)

- The storing operation of a CPU register in the stack is known as a **PUSH** and before pushing data SP is incremented by ONE automatically.
- Getting the contents from the stack back into a CPU register is called a **POP** and after retrieving data SP is decremented by ONE automatically.
- When the 8051 is initialized, the SP register contains the value 07H.
- This means that the RAM location 08H is the first location used for the stack to store data.

Pushing onto the Stack

Show the stack and stack pointer for the following. Assume the default stack area and register 0 is selected.

```
MOV    R6 , #25H
MOV    R1 , #12H
MOV    R4 , #0F3H
PUSH   6
PUSH   1
PUSH   4
```

Solution:

	After PUSH 6	After PUSH 1	After PUSH 4
0B			
0A			F3
09		12	12
08	25	25	25
Start SP = 07	SP = 08	SP = 09	SP = 0A

Popping from the Stack

Examining the stack, show the contents of the registers and SP after execution of the following instructions. All values are in hex.

```
POP    3      ; POP stack into R3
POP    5      ; POP stack into R5
POP    2      ; POP stack into R2
```

Solution:

	After POP 3	After POP 5	After POP 2
0B 54 0A F9 09 76 08 6C	0B 0A F9 09 76 08 6C	0B 0A 09 76 08 6C	0B 0A 09 08 6C
Start SP = 0B	SP = 0A	SP = 09	SP = 08

Stack and Bank 1 Conflict

Show the stack and stack pointer for the following instructions.

```
MOV    SP, #5FH    ;make RAM location 60H
                        ;first stack location
```

```
MOV    R2, #25H
MOV    R1, #12H
MOV    R4, #0F3H
PUSH   2
PUSH   1
PUSH   4
```

Solution:

	After PUSH 2	After PUSH 1	After PUSH 4
63	63	63	63
62	62	62	62 F3
61	61	61 12	61 12
60	60 25	60 25	60 25
Start SP = 5F	SP = 60	SP = 61	SP = 62

Machine cycles

Instruction and Machine cycle

Instruction Cycle:

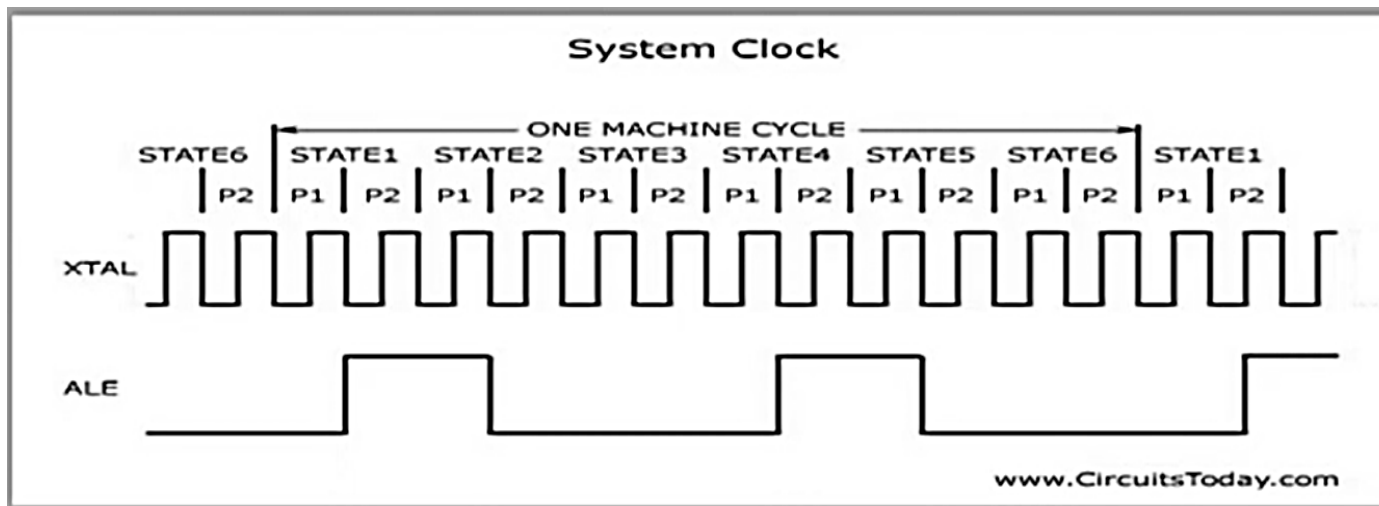
- Instruction execution consists of **fetching** of instruction, **decoding** the instruction and **executing** the instruction.
- Time required for completing the execution of an instruction by microcontroller is called instruction cycle.
- Instruction cycle consists of many machine cycle (1 to 4).

Machine Cycle:

- Time required to complete any simple instruction or part of complex instruction is called machine cycle.
- In 8051 one machine cycle consists of 6 states or **12 clock cycles**.

Instruction and Machine cycle

- Machine cycle of 8051 has six states (S1 - S6).
- Each state has two clock pulses (P1 and P2).
- Each Machine cycle has 12 clock pulses.
- Time period for 1 clock pulses = $1/\text{freq}$
- Time period for 1 Machine Cycle = $12 * 1/\text{freq}$



Instruction Time Calculation

- Depends on the clock frequency of oscillator
- 1 machine cycle = 12 oscillator cycle
- For a 11.0592MHz oscillator, the time for 1 machine cycle is:

$$T_{\text{machine cycle}} = \frac{1}{11.0592 \times 10^6} \cdot (12) = 1.085 \mu\text{s}$$

The following shows crystal frequency for three different 8051-based systems. Find the period of the machine cycle in each case.

(a) 11.0592 MHz (b) 16 MHz (c) 20 MHz

Solution:

(a) Period of machine cycle = $1 / 11.0592 \text{ MHz} * 12 = 1.085 \mu\text{s}$
(microsecond)

(b) Machine cycle (MC) = $1 / 16 \text{ MHz} * 12 = 0.75 \mu\text{s}$

(c) MC = $1 / 20 \text{ MHz} * 12 = 0.60 \mu\text{s}$

Example

For an 8051 system of 11.0592MHz, find how long it takes to execute each of the following instructions.

(a) MOV R3, #55

(b) DEC R3

(c) DJNZ R2, target (Decrement and Jump if Not Zero)
(d) LJMP and target is label

(e) SJMP

(f) NOP (No Operation)

(g) MUL AB

Solution

Instruction	Machine cycles	Time to execute
(a) MOV R3, #55	1	$1 \times 1.085\mu\text{s} = 1.085 \mu\text{s}$
(b) DEC R3	1	$1 \times 1.085\mu\text{s} = 1.085 \mu\text{s}$
(c) DJNZ R2, target	2	$2 \times 1.085\mu\text{s} = 2.17 \mu\text{s}$
(d) LJMP	2	$2 \times 1.085\mu\text{s} = 2.17 \mu\text{s}$
(e) SJMP	2	$2 \times 1.085\mu\text{s} = 2.17 \mu\text{s}$
(f) NOP	1	$1 \times 1.085\mu\text{s} = 1.085 \mu\text{s}$
(g) MUL AB	4	$4 \times 1.085\mu\text{s} = 4.34 \mu\text{s}$

Example

Machine Cycle

ORG	00h	--
MOV	A, #100	2
MOV	B, #2	2
MUL	AB	4
PUSH	01	1
MOV	A, B	2
PUSH	02	1
ADD	A, R1	1
MOV	30h, A	2
SWAP	A	1
END		--

It swaps the upper nibble (bits 7–4) with the lower nibble (bits 3–0) in the accumulator.

Total machine cycles required,

$$[2 + 2 + 4 + 1 + 2 + 1 + 1 + 2 + 1] = 16 \text{ machine cycles}$$

The execution time of the program segment is :

$$\text{Execution time} = 16 \times 1.085\mu\text{s} = 17.36\mu\text{s} \#$$

Example

Find the size of the delay in following program, if the crystal frequency is 11.0592MHz.

```
        MOV  A, #55H
AGAIN:  MOV  P1, A
        ACALL DELAY
        CPL  A
        SJMP AGAIN
;---time delay-----
DELAY:  MOV  R3, #200
HERE:   DJNZ R3, HERE
        RET
```

; Complement

A simple way to short jump to itself in order to keep the microcontroller busy

HERE: SJMP HERE

We can use the following:

SJMP \$

Activate
Go to Set

Machine cycle

DELAY: MOV R3, #200	1
HERE: DJNZ R3, HERE	2
RET	2

Therefore, $[(200 \times 2) + 1 + 2] \times 1.085 \mu s = 436.255 \mu s$.

ACALL

- The ACALL instruction is used for jumping to a specific subroutine address and storing the return address (the address of the instruction following the ACALL) onto the stack.
- The return address allows the program to return to the correct location after the subroutine execution is completed.
- The address range for ACALL is from **0x0000 to 0x07FF**, meaning it can call any subroutine within the first 2K of memory.
- The ACALL instruction pushes the current Program Counter (PC) (the return address) onto the stack.
- It then loads the Program Counter (PC) with the address of the subroutine specified by the address argument.
- After executing the subroutine, a RET (return) instruction is typically used to pop the return address from the stack and continue execution from that point.

For an 8051 system of 11.0592 MHz, find the time delay for the following subroutine:

		Machine Cycle
DELAY: MOV	R3,#250	1
HERE:	NOP	1
	NOP	1
	NOP	1
	NOP	1
	DJNZ R3,HERE	2
	RET	2

Solution:

The time delay inside the HERE loop is

$$\mathbf{[250(1 + 1 + 1 + 1 + 2)] \times 1.085 \mu s = 1500 \times 1.085 \mu s = 1627.5 \mu s.}$$

Adding the two instructions outside the loop we have

$$\mathbf{1627.5 \mu s + 3 \times 1.085 \mu s = 1630.755 \mu s.}$$

Example

For a machine cycle of $1.085 \mu\text{s}$, find the time delay in the following subroutine.

	Machine Cycle
DELAY: MOV R2,#200	1
AGAIN: MOV R3,#250	1
HERE: NOP	1
NOP	1
DJNZ R3,HERE	2
DJNZ R2,AGAIN	2
RET	2

- For the HERE loop, we have $(4 \times 250) \times 1.085 \mu\text{s} = 1085 \mu\text{s}$.
- The AGAIN loop repeats the HERE loop 200 times; therefore, we have $200 \times 1085 \mu\text{s} = 217000$, if we do not include the overhead.
- However, the instructions “MOV R3,#250” and “DJNZ R2,AGAIN” at the beginning and end of the AGAIN loop add $(3 \times 200 \times 1.085 \mu\text{s}) = 651 \mu\text{s}$ to the time delay.

- As a result, we have $217000 + 651 = 217651 \mu s = 217.651$ milliseconds for total time delay associated with the above DELAY subroutine.
- Notice that in the case of a nested loop, as in all other time delay loops, the time is approximate since we have ignored the first and last instructions in the subroutine.

Table: Clocks per Machine Cycle (MC) for Various 8051 Versions

Chip/Maker	Clocks per Machine Cycle
AT89C51 Atmel	12
P89C54X2 Philips	6
DS5000 Dallas Semi	4
DS89C430/40/50 Dallas Semi	1

Find the period of the machine cycle (MC) in each case if XTAL = 11.0592 MHz, and discuss the impact on performance.
(a) AT89C51 (b) P89C54X2 (c) DS5000 (d) DS89C4x0

Solution:

- (a) $MC = 1/11.0592 \text{ MHz} * 12 = 1.085 \mu\text{s} \text{ (microsecond)} = 1085 \text{ ns}$
- (b) $MC = 1/11.0592 \text{ MHz} * 6 = 0.5425 \mu\text{s} = 542 \text{ ns}$
- (c) $MC = 1/11.0592 \text{ MHz} * 4 = 0.36 \mu\text{s} = 360 \text{ ns}$
- (d) $MC = 1/11.0592 \text{ MHz} * 1 = 0.0904 \mu\text{s} = 90 \text{ ns}$

Comparison of 8051 and DS89C4x0 Machine Cycles

Instruction	8051	DS89C4x0
MOV R3,#value	1	2
DEC R _x	1	1
DJNZ	2	4
LJMP	2	3
SJMP	2	3
NOP	1	1
MUL AB	4	9

For an AT8051 and DS89C430/40/50 system of 11.0592 MHz, find how long it takes to execute each of the following instructions.

**(a) MOV R3,#55 (b) DEC R3 (c) DJNZ R2,target (d) LJMP
(e) SJMP (f) NOP (no operation) (g) MUL AB**

Solution:

Instruction	AT8051	DS89C430/40/50
(a) MOV R3,#55	1x1085 ns = 1085 ns	2x90 ns = 180 ns
(b) DEC R3	1x1085 ns = 1085 ns	1x90 ns = 90 ns
(c) DJNZ R2,..	2x1085 ns = 2170 ns	4x90 ns = 360 ns
(d) LJMP	2x1085 ns = 2170 ns	3x90 ns = 270 ns
(e) SJMP	2x1085 ns = 2170 ns	3x90 ns = 270 ns
(f) NOP	1x1085 ns = 1085 ns	1x90 ns = 90 ns
(g) MUL AB	4x1085 ns = 4340 ns	9x90 ns = 810 ns

Find the time delay for the loop section of the following subroutine if it is run on a DS89C430 chip, assuming a crystal frequency of 11.0592 MHz.

DS89C430 Machine Cycle

DELAY:	MOV	R3,#250	
HERE:	NOP		1
	NOP		1
	NOP		1
	NOP		1
	DJNZ	R3,HERE	4
	RET		

Solution:

The time delay inside the HERE loop is

$$[250(1 + 1 + 1 + 1 + 4)] \times 90 \text{ ns} = 2000 \times 90 \text{ ns} = 180 \mu\text{s}.$$

Comparing this with 8051, DS89C4x0 is about 9 times faster.

$$(1627 \mu\text{s} / 180 \mu\text{s} = 9)$$

Placing Code in Program ROM

ROM Address	Machine Language	Assembly Language
0000	7D25	MOV R5, #25H
0002	7F34	MOV R7, #34H
0004	7400	MOV A, #0
0006	2D	ADD A, R5
0007	2F	ADD A, R7
0008	2412	ADD A, #12H
000A	80FE	HERE: SJMP HERE

Address	Code
0000	7D
0001	25
0002	7F
0003	34
0004	74
0005	00
0006	2D
0007	2F
0008	24
0009	12
000A	80
000B	FE

ROM Example

Find the ROM memory address of each of the following 8051 chips.

(a) AT89C51 with 4KB (b) DS89C420 with 16KB (c) DS5000-32 with 32KB

Solution:

(a) 4K bytes of ROM memory space = 4×1024 bytes = 4096 bytes.

This maps to address locations of 0 to 4095 = 0000 to 0FFFH.

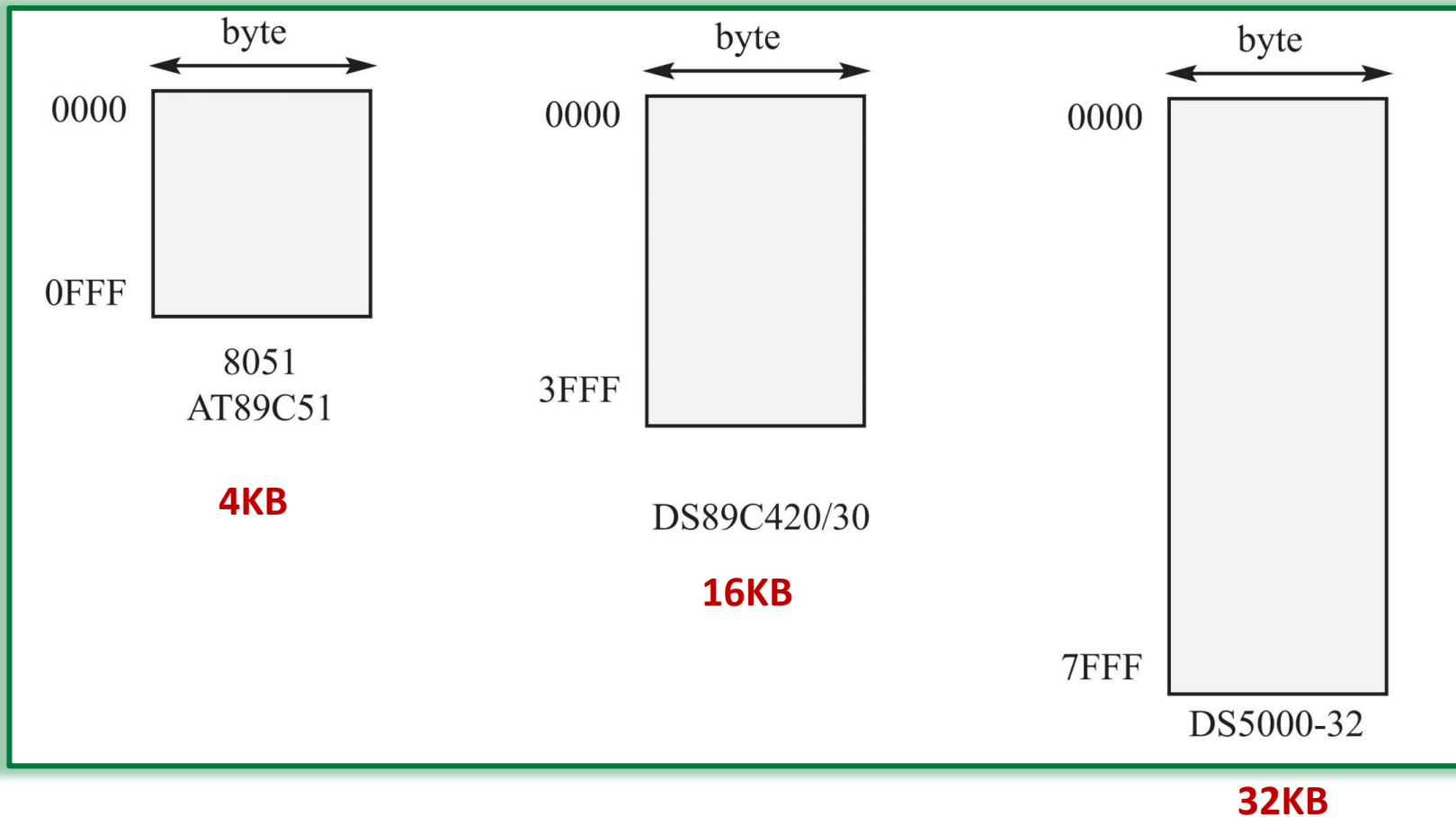
(b) 16K bytes of ROM memory space 16×1024 bytes = 16,384 bytes.

This maps to address locations of 0 to 16383 = 0000 to 3FFFH.

(c) 32K bytes = 32×1024 bytes = 32,768 bytes.

This maps to address locations of 0 to 32,767 = 0000 to 7FFFH.

ROM Memory Map in the 8051 Family



8051 On-Chip ROM Address Range

8051 Assembly Language Programming

Instruction Set of 8051

Structure of Assembly Language

An Assembly language instruction consists of four fields:

[label:] mnemonic [operands] [;comment]

- Brackets indicate that a field is optional, and not all lines have them. Brackets should not be typed in.
- An instruction typically has two main parts:
 - Opcode: Specifies the operation (e.g., add, subtract, load, store, jump).
 - Operands: Provide the data or the address of the data on which the operation will act.

Instruction Set of 8051

Instruction Set: Collection of instructions that the microcontroller is designed to execute.

8051 has simple instruction set in different groups.

There are:

- Data transfer instructions
- Arithmetic instructions
- Logical instructions
- Bit control instructions
- Branching and looping instructions

Data Transfer Instructions

- These instructions are used to transfer the data from source operand to destination operand.
- All the store, move, load, exchange input and output instructions belong to this group.
- MOV A, Rn ; Move Reg to Acc
- MOVX A,@DPTR ; Move external RAM to Accumulator
- PUSH direct ; PUSH direct byte on to stack
- POP direct ; POP direct byte from stack

Arithmetic instructions

These instructions are used to perform various mathematical operations like addition, subtraction, multiplication, and division etc.

- `ADD A, R1` ;Add the content of register1 to Accumulator
- `ADDC A,#2` ;Add 2 to accumulator with carry
- `SUBB A,R2` ;Subtract content of register2 from Accumulator
- `INC A` ;Increment accumulator
- `DEC A` ;Decrement accumulator
- `MUL AB` ;Multiply A and B
- `DIV AB` ;Divide A by B

Logical instructions

- The logical instructions are the instructions which are used for performing some operations like AND, OR, NOT, X-OR and etc., on the operands.
- ANL A, Rn ; AND register to accumulator
- ORL A, Rn ; OR register to accumulator
- XRL A, Rn ; Exclusive OR Reg to Acc
- CLR A ; Clear Accumulator
- CPL A ; Complement Accumulator

Bit Manipulation Instructions

- `Clr C` ; clears the carry flag
- `Setb C` ; Sets the carry flag

Branch and Looping Instructions

- These instructions are used for both branching as well as looping.
- These instructions include conditional & unconditional jump or loop instructions.

Looping operation in the 8051:

- On repeating a sequence of instructions, a certain number of times will result in the formation of loop.
- The looping operation is used for running the same set of subroutine inside a program number of times as per the requirement.

Conditional Jump Instruction:

- Consider the instruction **DJNZ register; label** is used for performing a loop operation.
- In this instruction, the register is decremented by 1; if this is not zero, then 8051 jumps to the target address referred by the label

Branch and Looping Instructions

Conditional Jump Instructions

- JC ; Jump if carry equal to one
- JNC ; Jump if carry equal to zero
- JB ; Jump if bit equal to one
- JNB ; Jump if bit equal to zero
- JBC ; Jump if bit equal to one and clear bit

- JZ ; Jump if A=Zero
- JNZ ; Jump if A not equal to zero
- DJNZ ; Decrement and Jump if not equal to zero.

Branch and Looping Instructions

Unconditional Jump Instructions

In 8051 there are two unconditional jumps. They are:

- SJMP ; Short jump
- LJMP ; Long jump

Data Transfer Instructions

MOV Instruction

MOV instruction copies data from one location to another.

MOV Instruction Format:

MOV destination, source ;copy source to dest

**This instruction tells the CPU
to move (in reality, copy) the source operand to the destination
operand.**

DATA MOVING OR HANDLING INSTRUCTIONS

Mnemonics	Operational descriptions	Addressing modes	No. of cycles used	No. of bytes occupied
Mov a,#num	This instruction Copy the immediate data number in to acc	immediate	1	2
Mov Rx,a	This instruction Copy the data from acc to Rx	register	1	1
Mov a,Rx	This instruction Copy the data from Rx to acc	register	1	1
Mov Rx,#num	This instruction Copy the immediate data num in to Rx	immediate	1	2
Mov a,add	This instruction Copy the data from direct address location add to acc	direct	1	2
Mov add,a	This instruction Copy the data from acc to direct address add	direct	1	2
Mov add.#num	This instruction Copy the immediate data num in to direct address	direct	2	3

DATA MOVING OR HANDLING INSTRUCTIONS

Mnemonics	Operational descriptions	Addressing modes	No. of cycles used	No. of bytes occupied
Mov add1,add2	This instruction Copy the data from add2 to add1	direct	2	3
Mov Rx,add	This instruction Copy the data from direct	direct	2	2
Mov add,Rx	This instruction Copy the data from Rx to direct address add	direct	2	2
Mov @Rp,a	This instruction Copy the data in acc to address in Rp	Indirect	1	2
Mov a,@Rp	This instruction Copy the data that is at address in Rp to acc	Indirect	1	1
Mov add,@Rp	This instruction Copy the data that is at address in Rp to add	Indirect	2	2
Mov @Rp,add	This instruction Copy the data in add to address in Rp	Indirect	2	2

DATA MOVING OR HANDLING INSTRUCTIONS

Mnemonics	Operational descriptions	Addressing modes	No. of cycles used	No. of bytes occupied
Mov @Rp,#num	This instruction Copy the immediate byte num to the address in Rp	Indirect	1	2
Movx a,@Rp	This instruction Copy the content of external add in Rp to acc	Indirect	2	1
Movx a,@DPTR	This instruction Copy the content of external add in DPTR to acc	Indirect	2	1
Movx @Rp,a	This instruction Copy the content of acc to the external add in Rp	Indirect	2	1
Movx @DPTR,a	This instruction Copy the content of acc to the external add in DPTR	Indirect	2	1
Movc a,@a+DPTR	In this the address of instruction is formed by adding acc and DPTR and their content is copied to acc	indirect	2	1
Movc a,@a+PC	In this the address of instruction is formed by adding acc and PC and its content is copied to acc	indirect	2	1

DATA MOVING OR HANDLING INSTRUCTIONS

Mnemonics	Operational descriptions	Addressing modes	No. of cycles used	No. of bytes occupied
Push add	In this instruction Increment Stack Pointer (SP) and copy the data from source add to internal RAM address contained in SP	Direct	2	2
Pop add	This instruction copy the data from an internal RAM address contained in SP to destination add and decrement SP	direct	2	2
Xch a, Rx	This instruction Exchange the data between acc and Rx	Register	1	1
Xch a, add	This instruction Exchange the data between acc and given add	Direct	1	2
Xch a,@Rp	This instruction Exchange the data between acc and address in Rp	Indirect	1	1
Xchd a, @Rp	This instruction Exchange only lower nibble of acc and address in Rp	indirect	1	1

Arithmetic instructions

ADD Instruction

ADD Instruction Format:

**ADD A,source ;ADD the source operand
;to the accumulator**

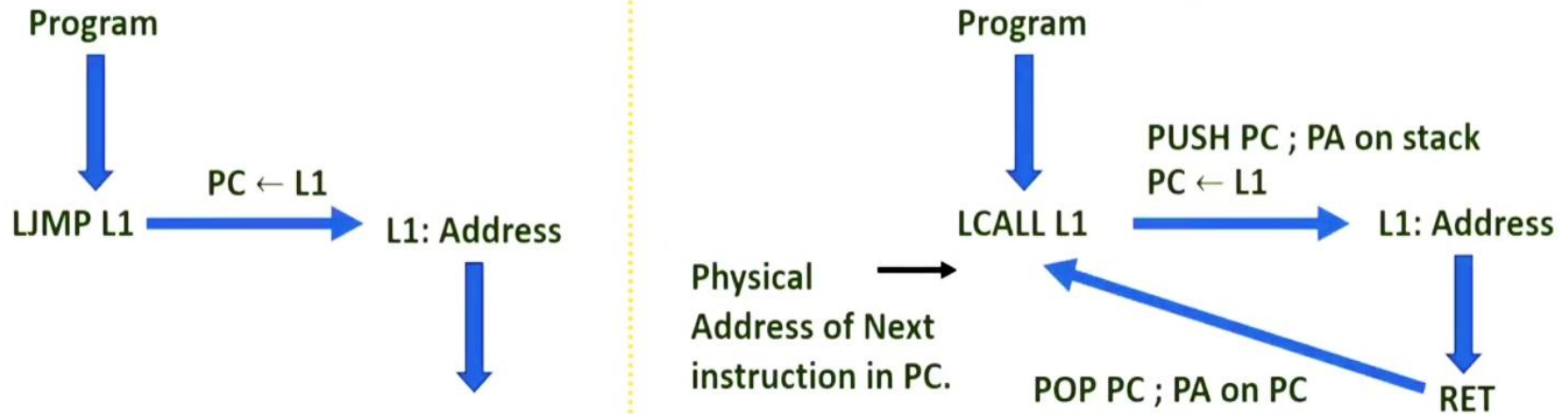
**The ADD instruction tells the CPU
to add the source byte to register A
and put the result in register A.**

Branch and Looping Instructions

LOOP, JUMP and CALL Instructions

Instruction	Action
JZ	Jump if A = 0
JNZ	Jump if A $\neq$ 0
DJNZ	Decrement and jump if register $\neq$ 0
CJNE A, data	Jump if A $\neq$ data
CJNE reg, #data	Jump if byte $\neq$ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

JMP and CALL of 8051 μ C



JMP	CALL
❖ Here, we just jump to new location and then we move on with given program.	❖ Here, we jump to new location for subroutine, after subroutine we come back to that location again.
❖ No need of retrieve back to branch location.	❖ By RET at the end of subroutine, it will retrieve back to branch location.
❖ JMP doesn't store branch location.	❖ CALL stores branch location on Stack.
❖ In 8051, JMP is of three types:	❖ In 8051, CALL is of two types:
❖ SJMP, AJMP and LJMP	❖ ACALL and LCALL
❖ In 8051, SJMP can be conditional and unconditional.	❖ In 8051, we have only unconditional CALL instructions.
❖ JMP is used for program logic only.	❖ CALL is used for function execution in main program.

Branch Operations of 8051 μ C

{SJMP, AJMP & LJMP}

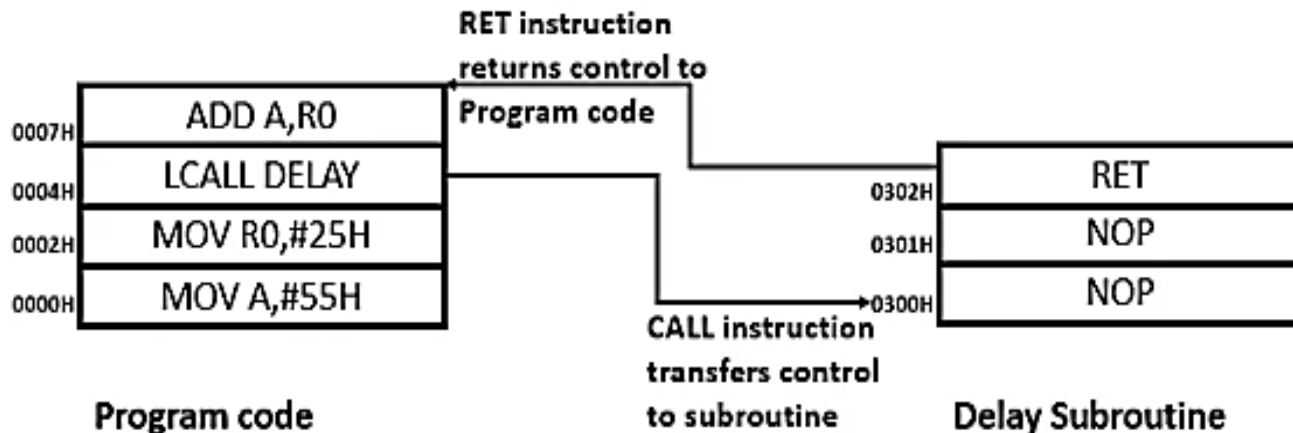
Parameters	SJMP	AJMP	LJMP
Name	▪ SJMP – Short Jump	▪ AJMP – Absolute Jump	▪ LJMP – Long Jump
Syntax	▪ SJMP Label1	▪ AJMP Label2	▪ LJMP Label3
Range	▪ -128 to +127 locations, because of Label1 is 8 bits signed No.	▪ Maximum range is 2KB.	▪ Maximum range is 64KB.
Size	▪ 2 Bytes {1 Opcode + 1 Label}	▪ 2 Bytes {1 Opcode + 1 Label}	▪ 3 Bytes {1 Opcode + 2 Label}
Address calculation	▪ PC = PC of Next instruction + Label1	▪ PC = 1 st 5 bits same of PC + 3bits of AJMP + 8 bits of Label2	▪ PC = 16 bits Label3
Instructions	▪ SJMP ▪ All conditional Jump available	▪ AJMP ▪ ACALL	▪ LJMP ▪ LCALL
Usage	▪ Generally, we use it in same program jump with many conditions and unconditional.	▪ We use it for jump within same page of 2KB. ▪ 64Kb is bisected into 32 pages of 2KB	▪ We use it to jump anywhere in 64KB of 8051 μ C.

Branching Instructions in 8051

- A microcontroller sequentially executes instructions but, in some cases, transferring this control to another block of code becomes essential
- The branching instructions in the 8051 microcontroller are responsible for performing this operation.
- Tasks like **looping, calling delays, and conditional execution** of code can be performed using these branching instructions.

Call and Return instructions in 8051

- A microcontroller needs to perform the same tasks multiple numbers of times across the program, such as generating a delay. A subroutine is responsible for performing these repetitive tasks. Using subroutines saves memory and makes the program more efficient.
- Call instruction is used to transfer the control from the presently executing program code to the subroutine, and the return instruction is used to return the control to the program code



Call and Return instructions in 8051

Operation	Mnemonics	Description
Call	ACALL Address11	Calls a subroutine in the maximum address range of 2K bytes
	LCALL Address16	Calls a subroutine in the maximum address range of 64K bytes
Return	RET	Returns the control from subroutine
	RETI	Returns the control from an interrupt subroutine

LCALL Instruction: NO FLAGS AFFECTED

Opcode	Operand	Description	Size	Execution Time
LCALL	ADDRESS(16 BIT)	Transfers the control to the specified address(64KB)	3 bytes	24 clock cycles

LCALL instruction can access an address of 16 bits. Due to this reason, this instruction can access any memory location in the ROM space, but this instruction takes up 3 bytes of space and can waste memory resources. To save memory, the ACALL instruction is used.

ACALL Instruction: NO FLAGS AFFECTED

Opcode	Operand	Description	Size	Execution Time
ACALL	ADDRESS(11 BIT)	Transfers the control to the specified address restricted to a page of 2KB	2 bytes	24 clock cycles

The ACALL instruction is 2 bytes in size and can be used to access any address in a 2KB page. This instruction affects only 11 bits or the program counter as compared to 16 in the case of LCALL

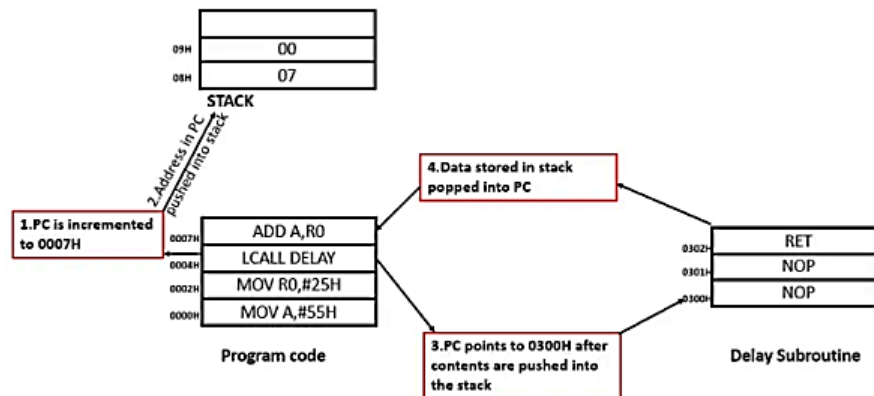
RET Instruction: NO FLAGS AFFECTED

Opcode	Operand	Description	Size	Execution Time
RET	NONE	Returns control to the program code	1 byte	24 clock cycles

The RET instruction is used to return from interrupt subroutines and works in the same way as the RET instruction if used outside a subroutine. When used inside a subroutine, RETI first enables interrupts of equal and lower priorities to the interrupt that it is used in. The program execution continues at the address that is calculated by popping the topmost 2 bytes off the stack. The most-significant-byte is popped off the stack first, followed by the least-significant-byte

Stack and control transfer

- The stack of the 8051 plays a crucial role in the transfer of control when it comes to call instructions.
- When the call instruction executes, the program counter increments so that it points to the next instruction. The contents of the program counter are pushed into the stack (lower byte first).
- After this, the program counter is loaded with the starting instruction of the subroutine transferring the control
- Once the RET instruction is encountered, the contents stored in the stack are popped back into the program counter, and the microcontroller starts executing the program code from where it had left it



Jumps in 8051

- Jump instruction is also used to transfer control in the 8051 microcontroller. But unlike a Call instruction, it does not call a subroutine and jumps to an address in the same program memory. Jumps in the 8051 microcontroller are used to perform looping and conditional execution of program code
- **Unconditional jumps-** These jumps do not evaluate a condition to transfer the control to another address.
- **Conditional jumps-** These jumps evaluate a particular condition to transfer the control to another address in the program code. All conditional jumps are short jumps

Jumps in 8051

AJMP Address11	Jumps to an address in a 2KB range	CJNE A, Direct address, Relative address	Jumps to relative address when accumulator=data stored at a direct address
LJMP Address16	Jumps to an address in a 64KB range	CJNE A, #Data,Relative address	Jumps to relative address when accumulator=data given by the programmer
SJMP Relative address	Jumps to an address in a 256-byte range (0 to 127 (0-7FH) range and -1 to -128 (FFH-80H).	CJNE @Rn, #Data,Relative address	Jumps to relative address when data at memory location stored in register=data given by the programmer
JMP @A+DPTR	[DPTR]<-[DPTR+A]	DJNZ Rn, Relative address	Decrements value in Rn and jump to relative address till Rn!=0
JZ Relative address	Jumps to address when accumulator=0		
JNZ Relative address	Jumps to address when accumulator!=0		

DJNZ Direct address, Relative address	Decrements value at memory location stored in a register and jump to relative address till memory location stored in register =0
--	--

Unconditional Jump

Opcode	Operand	Description	Size	Execution Time
LJMP	ADDRESS(16 BITS)	Transfers control within program code. The maximum range of this jump is 64KB	3 bytes	24 clock cycles

AJMP	ADDRESS(11 BITS)	Transfers control within program code. The maximum range of this jump is 2KB	2 bytes	24 clock cycles
------	------------------	--	---------	-----------------

SJMP	Relative Address	Uses the given address as an offset to the executing address and transfers the control to the new address. The maximum range of 256 bytes	2 bytes	24 clock cycles
------	------------------	---	---------	-----------------

NO FLAGS AFFECTED

Conditional Jump

Opcode	Operand	Description	Size	Time
JZ	Relative Address	Uses the given address as an offset to the executing address and transfers the control to the new address if the value in the accumulator=0. The maximum range of 256 bytes	2 bytes	24 clock cycles

JNZ	Relative Address	Uses the given address as an offset to the executing address and transfers the control to the new address if the value in the accumulator !=0. The maximum range of 256 bytes	2 bytes	24 clock cycles
-----	------------------	---	---------	-----------------

NO FLAGS AFFECTED

Conditional Jump

CJNE

A, Direct address, Relative address	Compares the data stored in the accumulator with the data stored at the direct address. If the values are the same, then the control is transferred to the relative address	3 bytes	24 clock cycles	@Rn, #Data, Relative address	Compares the data stored at the address stored in the register with the data given by the programmer. If the values are the same, then the control is transferred to the relative address	3 bytes	24 clock cycles
@Rn, #Data, Relative address	Compares the data stored at the address stored in the register with the data given by the programmer. If the values are the same, then the control is transferred to the relative address	3 bytes	24 clock cycles				

Conditional Jump

DJNZ

Rn, Relative address	This instruction is used for looping in 8051. It decrements the value stored in Rn and jumps to the relative address till the value in the register!=0	2 bytes	24 clock cycles
Direct address, Relative address	This instruction is used for looping in 8051. It decrements the value stored at the given address and jumps to the relative address till the value in the address!=0	3 bytes	24 clock cycles

NO FLAGS AFFECTED

Looping in the 8051

Write a program to

(a) clear ACC, then

(b) add 3 to the accumulator ten times.

Solution:

;This program adds value 3 to the ACC ten times

	MOV A,#0	;A=0, clear ACC
	MOV R2,#10	;load counter R2=10
AGAIN:	ADD A,#03	;add 03 to ACC
	DJNZ R2,AGAIN	;repeat until R2=0(10 times)
	MOV R5,A	;save A in R5

What is the maximum number of times that the loop in the previous Example can be repeated?

Solution:

Since R2 holds the count and R2 is an 8-bit register, it can hold a maximum of FFH (255 decimal); therefore, the loop can be repeated a maximum of 256 times.

Loop Inside a Loop

Write a program to (a) load the accumulator with the value 55H, and (b) complement the ACC 700 times.

Solution:

Since 700 is larger than 255 (the maximum capacity of any register), we use two registers (R2 and R3) to hold the count.

	MOV	A,#55H	;A=55H
	MOV	R3,#10	;R3=10, the outer loop count
NEXT:	MOV	R2,#70	;R2=70, the inner loop count
AGAIN:	CPL	A	;complement A register
	DJNZ	R2,AGAIN	;repeat it 70 times (inner loop)
	DJNZ	R3,NEXT	

In this program, R2 is used to keep the inner loop count.

In the instruction “DJNZ R2,AGAIN”, whenever R2 becomes 0 it falls through and “DJNZ R3,NEXT” is executed. This instruction forces the CPU to load R2 with the count 70 and the inner loop starts again. This process will continue until R3 becomes zero and the outer loop is finished.

Write a program to determine if R5 contains the value 0. If so, put 55H in it.

Solution:

```
MOV  A,R5           ;copy R5 to A
JNZ  NEXT           ;jump if A is not zero
MOV  R5,#55H
```

NEXT: ...

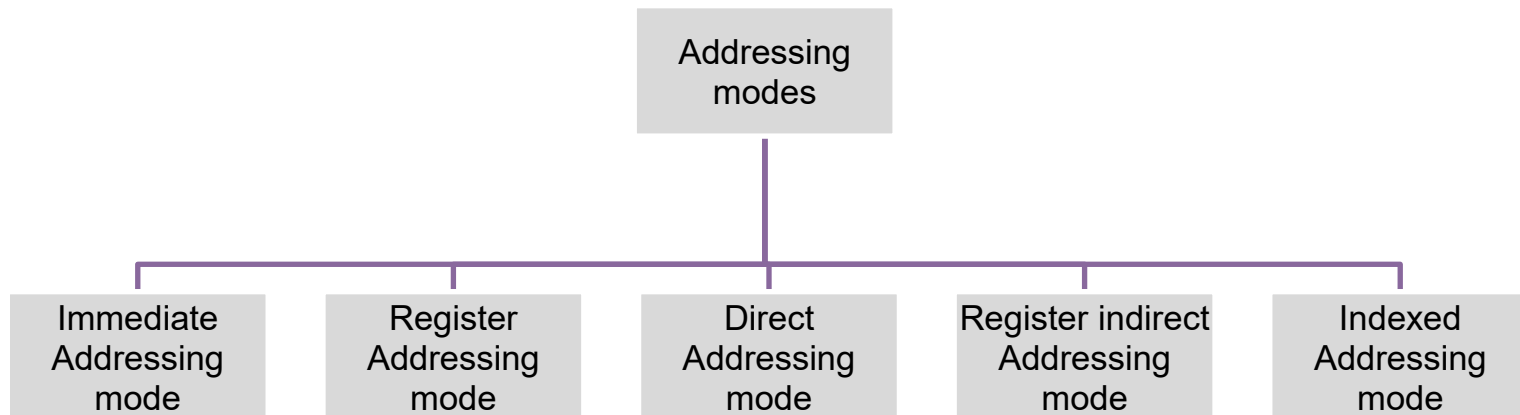
8051

ADDRESSING MODES

Addressing Modes

The various formats of specifying the operands are called addressing modes.

Types of Addressing modes



Immediate Addressing Mode

- ❑ In this Addressing mode, data (1byte/2bytes) specified in instruction itself.
- ❑ Data is specified by '#' symbol before data in the instruction.

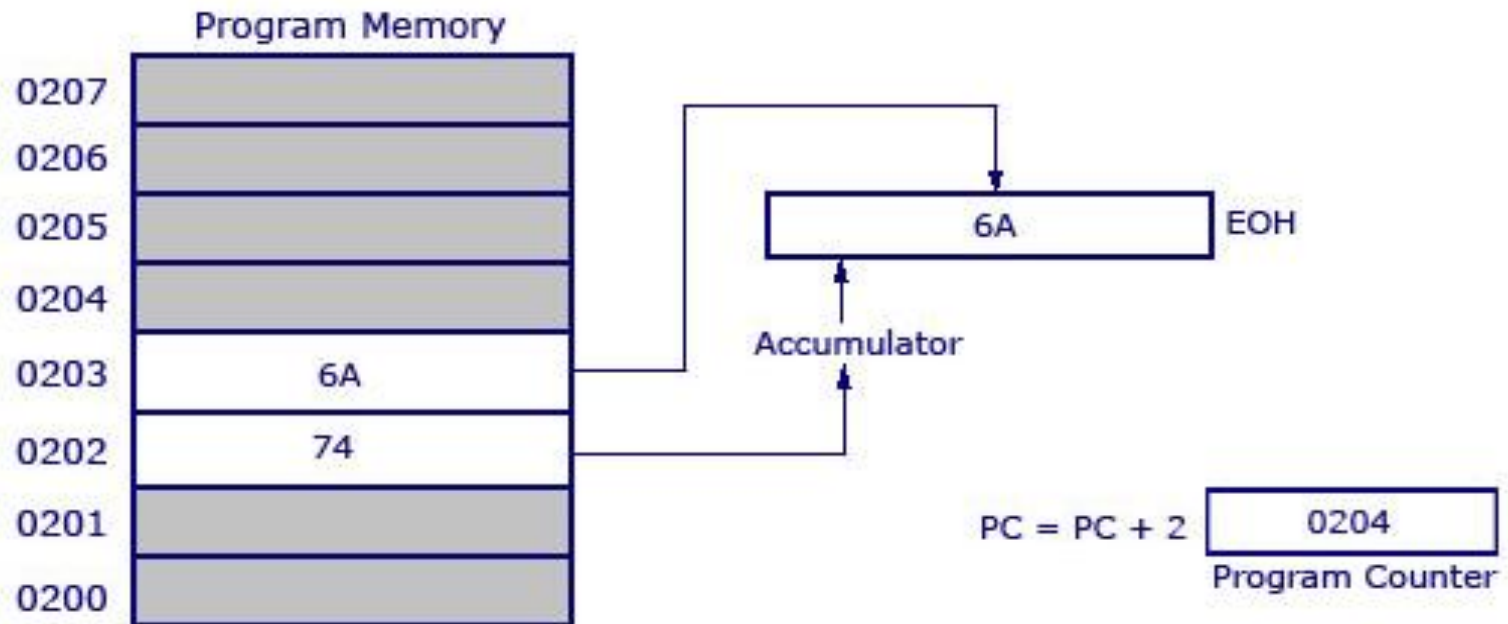
Example:

```
MOV A,#15H      ; A ← 15H  
MOV DPTR,#1000H ; DPTR ← 1000H
```

- MOV A, #6AH
- This addressing mode is named as “**immediate**” because it transfers an 8-bit data immediately to the accumulator (destination operand).

Immediate Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, #6AH	74H	2	1



www.CircuitsToday.com

Register Addressing Mode

MOV A, R4

- ✓ Register Addressing Mode
- ❑ In this Addressing mode, data is specified by registers in instruction.
- ❑ The permitted registers are A, R7, R6, ... , R0.

Example:

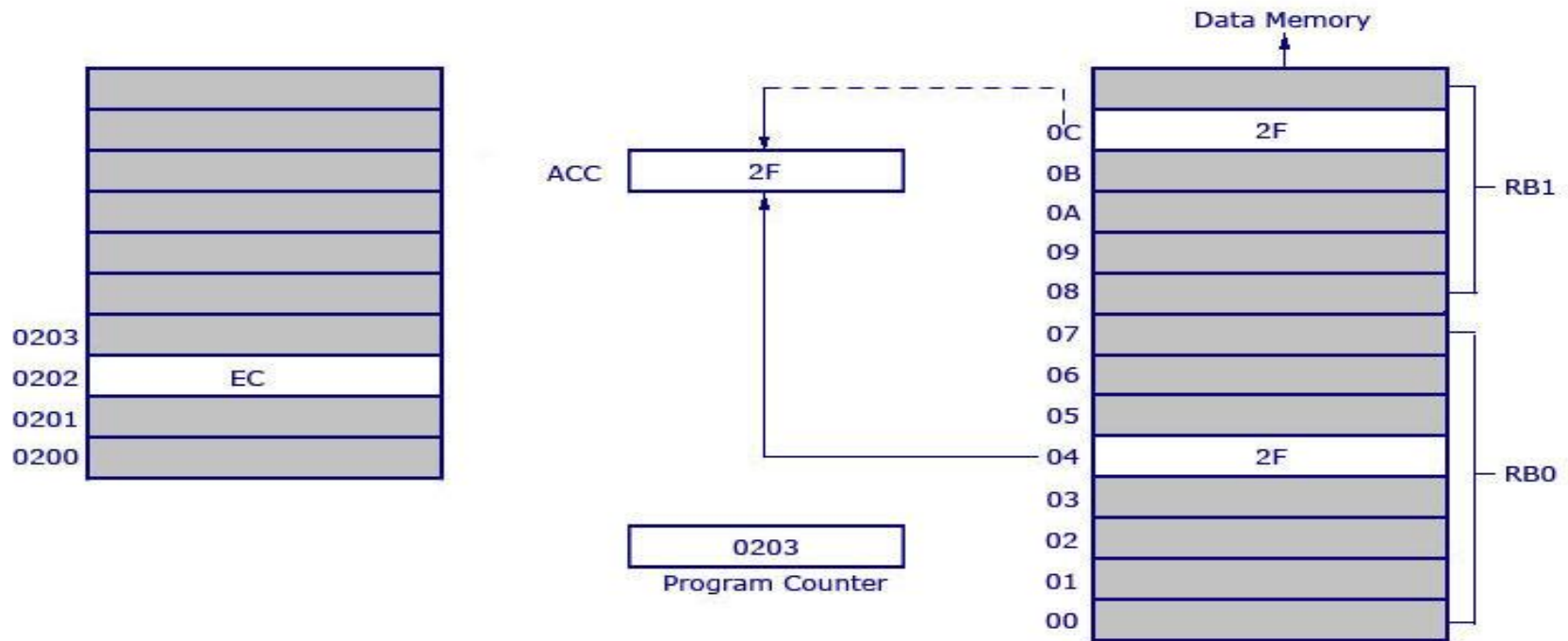
MOV A,R2	; A ← R2
MOV R2,A	; R2 ← A
MOV R1,R2	; Not Allowed with 8051,

Register Addressing Mode

MOV A, R4

Register Direct Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, R4	ECH	1	1



www.CircuitsToday.com

Direct Addressing Mode

- ❑ In this Addressing mode, address of operand is given in instruction.
- ❑ Only Internal RAM and SFR address are allowed.

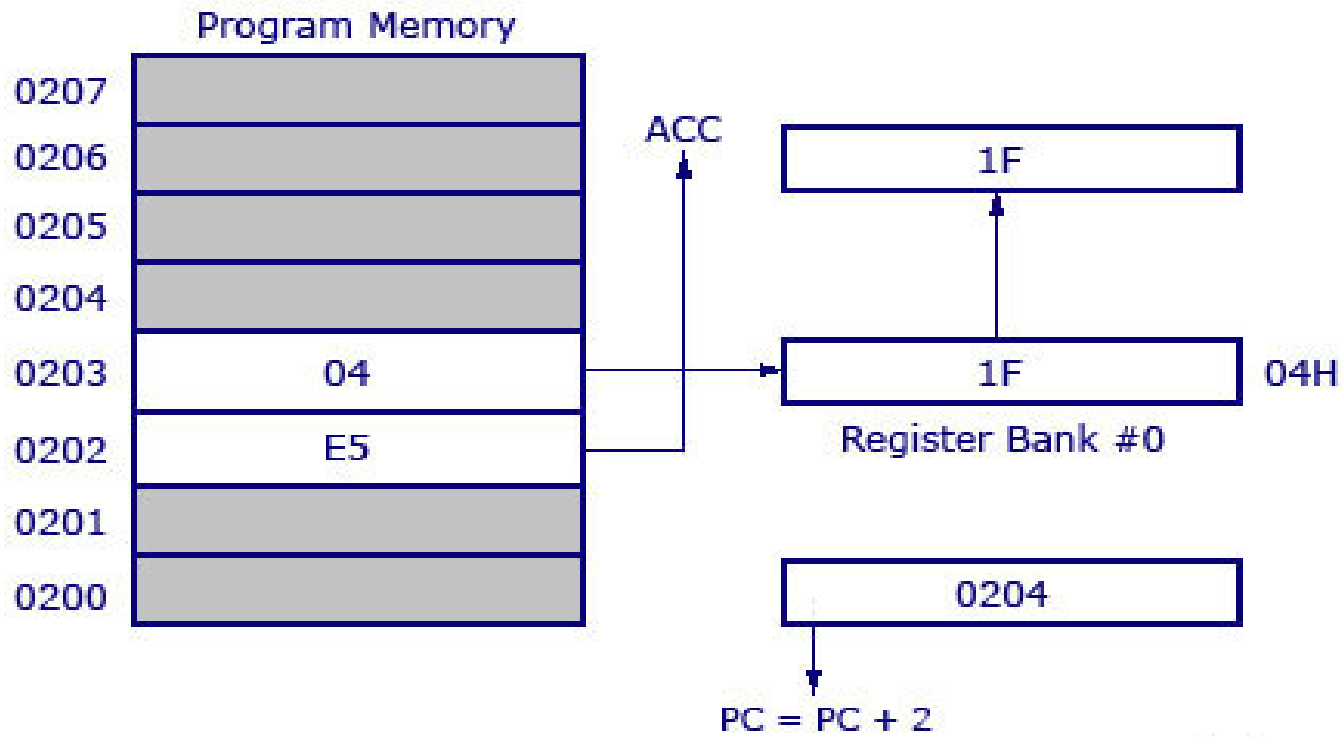
Example:

MOV A,35H	; A ← [35H]
MOV A,80H	; A ← [80H], Content of port 0 is [80H]
MOV 30H, 35H	; [30H] ← [35H]

- **MOV A, 04H**
- Here 04H is the address of register 4 of register bank0.
- When this instruction is executed, what ever data is stored in register 04H is moved to accumulator.

Direct Addressing Mode

Instruction	Opcode	Bytes	Cycles
MOV A, 04H	E5	2	1



Register Indirect Addressing Mode

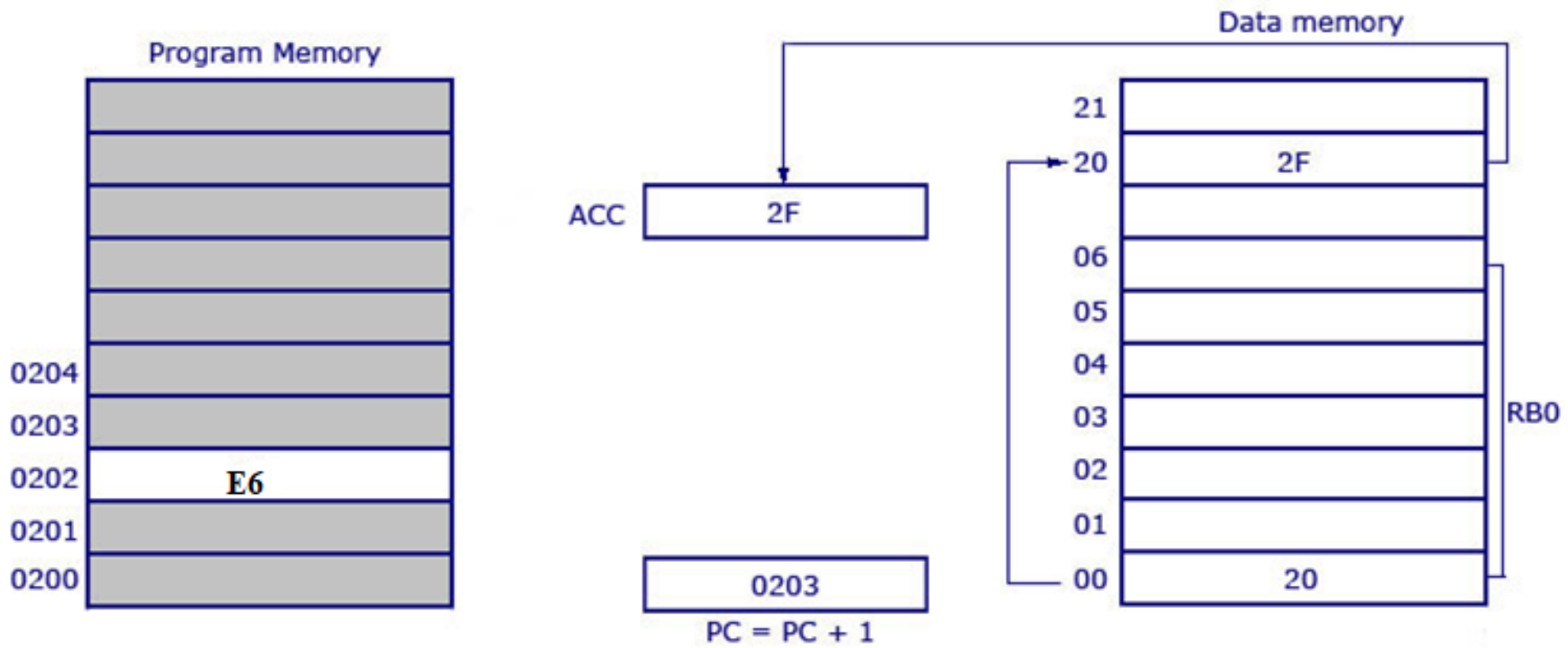
MOV A, @R0

- ❑ In this Addressing mode, address of operand will be given by register.
- ❑ Internal and External RAM can be accessed by this mode.
 - ❖ Internal RAM with 8bits of addressing, "@" is used here
MOV A,@R1 ; A ← [R1]
 - ❖ External RAM with 16bits of addressing by DPTR, "X" is used here
MOVX A,@DPTR ; A ← [DPTR]
MOVX @DPTR,A ; [DPTR] ← A
 - ❖ External RAM with 8bits of addressing by R0 or R1
MOVX A,@R1 ; A ← [R1], If R1 is 25H then
[R1] = [0025H]
MOVX @R0,A ; [R0] ← A, If R0 is 35H then
[R0] = [0035H]

Register Indirect Addressing Mode

MOV A, @R0

Instruction	Opcode	Bytes	Cycles
MOV A, @ R0	E6H	1	1



Indexed Addressing Mode

MOVC A, @A+DPTR

- ❑ This addressing mode is used to access data from code memory {Internal ROM or External ROM}.
- ❑ In instruction, we use 'C' to operate with Indexed Addressing mode.

Example:

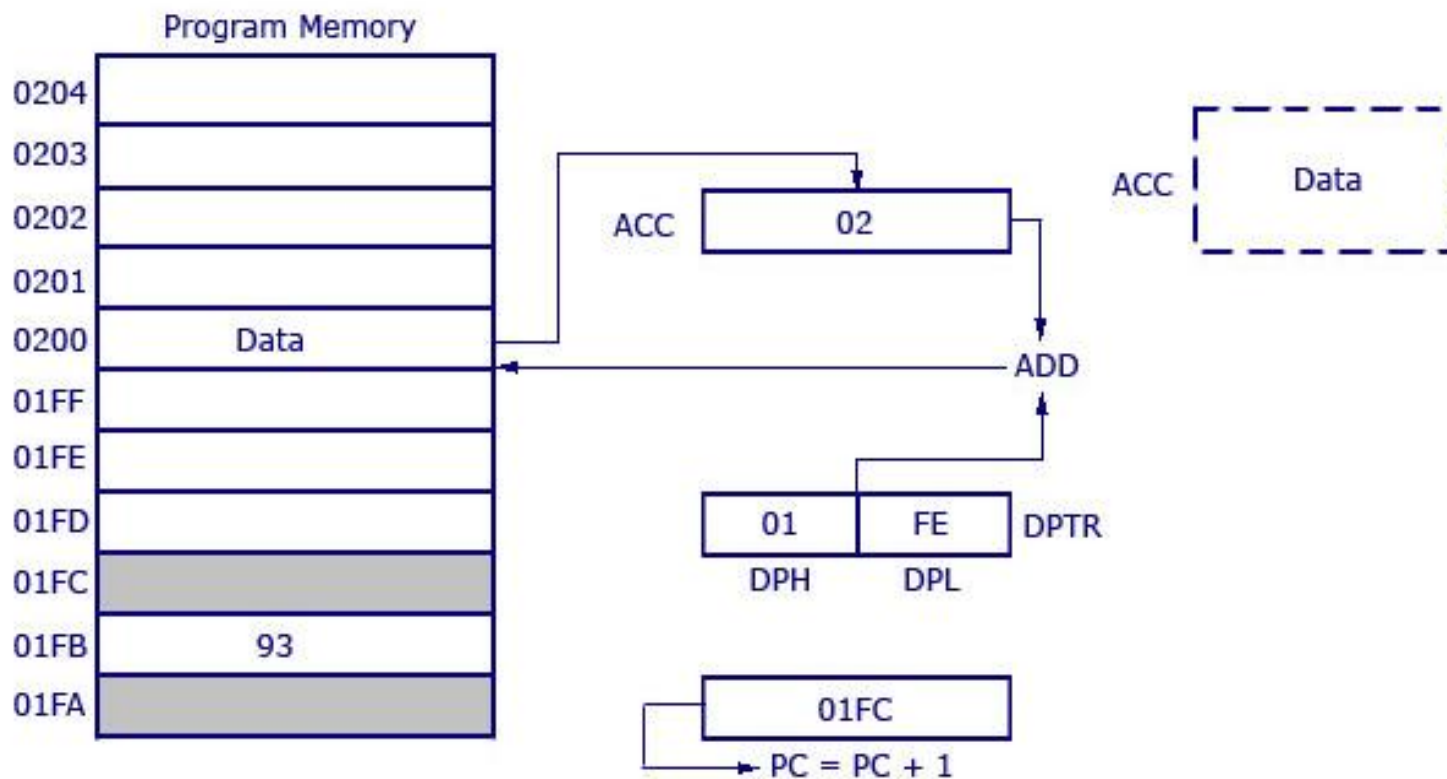
MOVC A,@A+DPTR ; $A \leftarrow [A+DPTR]$

MOVC A,@A+PC ; $A \leftarrow [A+PC]$

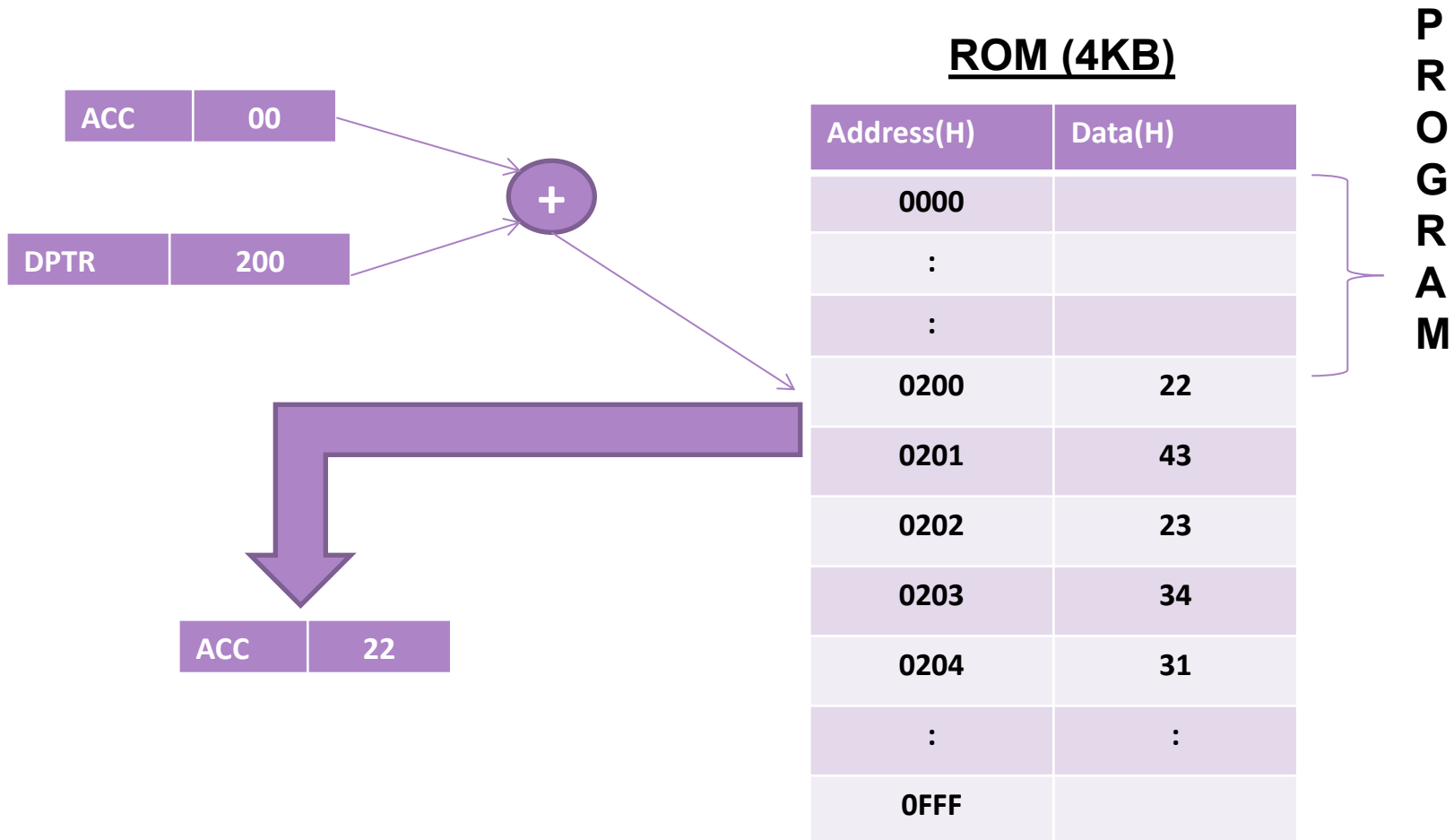
Indexed Addressing Mode

MOVC A, @A+DPTR

Instruction	Opcode	Bytes	Cycles
MOVC A, @A + DPTR	93H	1	2



MOVC A,@A+DPTR



Example

- MOV R0,#25H
- MOV 25H,#04H
- MOV A,#54H
- ADD A, @R0
- MOV 7FH, A
- MOV 4FH,25H

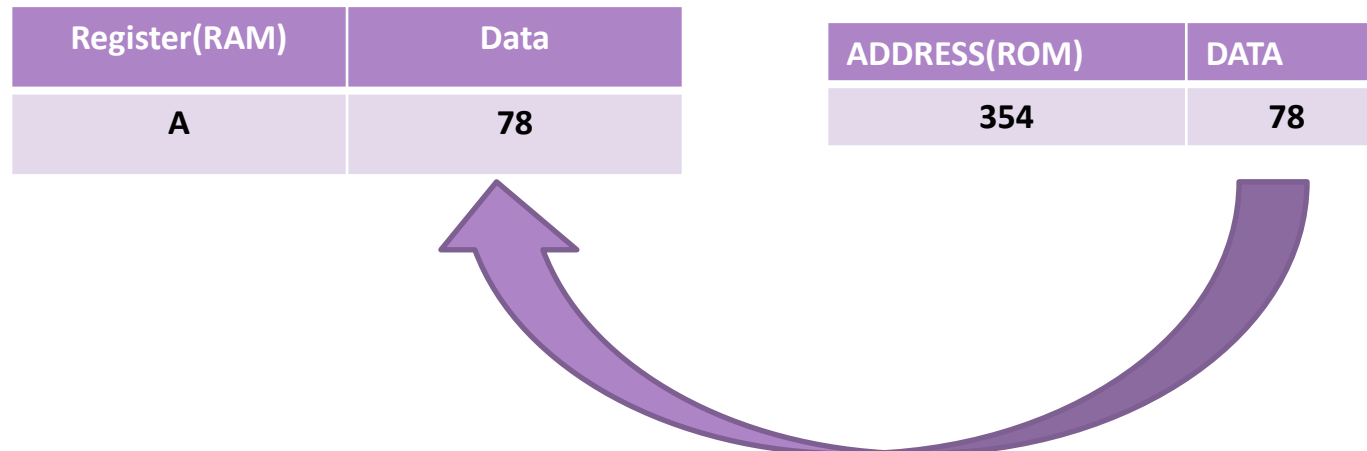
Address(RAM)	data
7F	58
4FH	04
25H	04
00 (R0 REG)	25

After execution of add instruction

Address(RAM)	data
E0 (A REGISTER)	58

Example

- MOV A,#54H ; OR MOV E0H,#54H
- MOV DPTR,#300H
- MOVC A,@A+DPTR ; AFTER A=



Example

- MOV A,#02H
- MOV R1,#65H
- MOV 65H,#02H
- ADD A,@R1

ADDRESS(RAM)	DATA
65H	02
:	
01H(R1 REG)	65

After execution of add instruction

ADDRESS	DATA
E0 (A REG)	04

Example

- ADD A,65H ; Direct addressing mode
- ADD A,#65H ; Immediate addressing mode
- ADD A,@R1 ; register indirect addressing mode
- ADD A, R1 ; register addressing mode
- SUBB A,R7 ; register addressing mode
- SUBB A,@R7 ; INVALID instruction; only R0 and R1 Register should be used for indirect addressing
- MUL A,B ; INVALID instruction
- MUL R0R1 ; INVALID instruction
- MOV #45h,54H ; INVALID instruction

Find the sum of the values 79H, F5H, and E2H. Put the sum in registers R0 (low byte) and R5 (high byte).

Solution:

	MOV	A,#0	;clear A(A=0)
	MOV	R5,A	;clear R5
	ADD	A,#79H	;A=0+79H=79H
	JNC	N_1	;if no carry, add next number
	INC	R5	;if CY=1, increment R5
N_1:	ADD	A,#0F5H	;A=79+F5=6E and CY=1
	JNC	N_2	;jump if CY=0
	INC	R5	;If CY=1 then increment R5(R5=1)
N_2:	ADD	A,#0E2H	;A=6E+E2=50 and CY=1
	JNC	OVER	;jump if CY=0
	INC	R5	;if CY=1, increment 5
OVER:	MOV	R0,A	;Now R0=50H, and R5=02

8051 Assembly Main Program That Calls Subroutines

;MAIN program calling subroutines

ORG 0

MAIN: LCALL SUBR_1

LCALL SUBR_2

HERE: SJMP HERE

;—————end of MAIN

SUBR_1:....

....

RET

;—————end of subroutine 1

SUBR_2:....

....

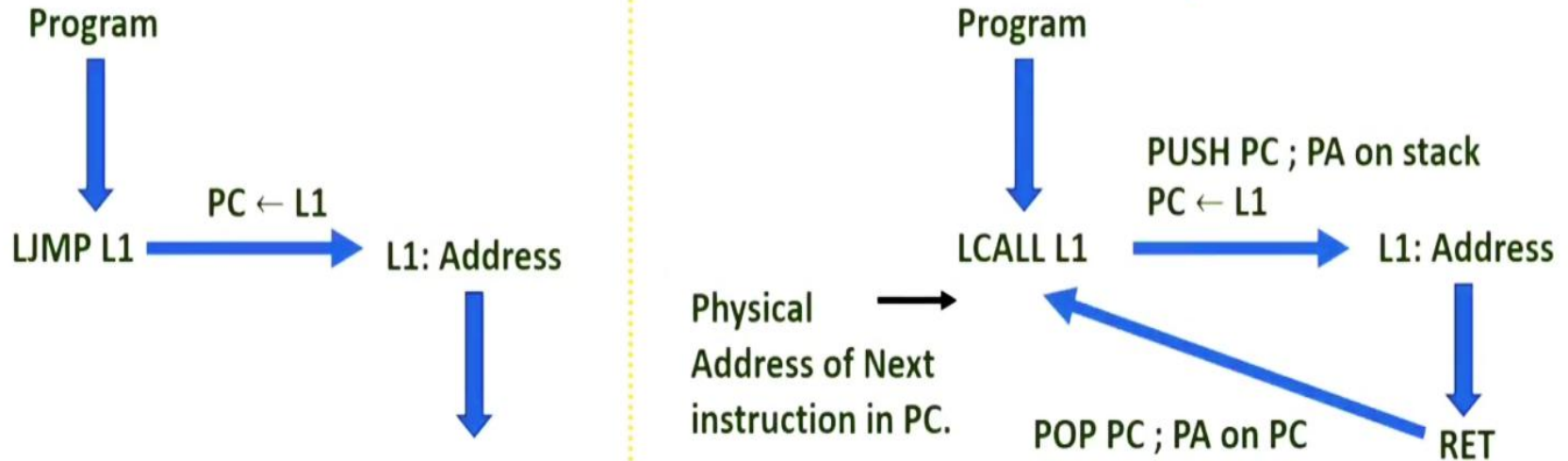
RET

;—————end of subroutine 2

END

;end of the asm file

JMP and CALL of 8051 μ C



JMP	CALL
❖ Here, we just jump to new location and then we move on with given program.	❖ Here, we jump to new location for subroutine, after subroutine we come back to that location again.
❖ No need of retrieve back to branch location.	❖ By RET at the end of subroutine, it will retrieve back to branch location.
❖ JMP doesn't store branch location.	❖ CALL stores branch location on Stack.
❖ In 8051, JMP is of three types: ❖ SJMP, AJMP and LJMP	❖ In 8051, CALL is of two types: ❖ ACALL and LCALL
❖ In 8051, SJMP can be conditional and unconditional.	❖ In 8051, we have only unconditional CALL instructions.
❖ JMP is used for program logic only.	❖ CALL is used for function execution in main program.

Analyze the stack contents after the execution of the first LCALL in the following.

```

001      0000                      ORG  0
002      0000      7455      BACK:  MOV  A,#55H      ;load A with 55H
003      0002      F590                      MOV  P1,A      ;send 55H to port 1
004      0004      120300                     LCALL DELAY      ;time delay
005      0007      74AA                      MOV  A,#0AAH      ;load A with AAH
006      0009      F590                      MOV  P1,A      ;send AAH to port 1
007      000B      120300                     LCALL DELAY
008      000E      80F0                      SJMP  BACK      ;keep doing this
009      0010
010      0010      ;—————this is the delay subroutine
011      0300                      ORG 300H
012      0300                      DELAY:
013      0300      7DFF                      MOV  R5,#0FFH      ;R5=255
014      0302      DDFE      AGAIN:  DJNZ  R5,AGAIN      ;stay here
015      0304      22                      RET      ;return to caller
016      0305                      END      ;end of asm file

```

- LCALL instruction pushes the return address (the address of the next instruction after LCALL) onto the stack and transfers control to the specified subroutine.
- Initial SP value: 07H.
- When the LCALL DELAY is executed, the stack grows:
 - The low byte of the return address (07H, lower byte of 0007H) is pushed.
 - The high byte of the return address (00H, higher byte of 0007H) is pushed.
- SP after LCALL: $07H + 2 = 09H$.
- Stack contents:
 - At address 08H: 07H (low byte of 0007H).
 - At address 09H: 00H (high byte of 0007H).

0A	
09	00
08	07
SP = 09	

Analyze the stack for the first LCALL instruction in the following program.

01	0000		ORG	0	
02	0000	7455	BACK:	MOV	A,#55H ;load A with 55H
03	0002	F590		MOV	P1,A ;send 55H to port 1
04	0004	7C99		MOV	R4,#99H
05	0006	7D67		MOV	R5,#67H
06	0008	120300		LCALL	DELAY ;time delay
07	000B	74AA		MOV	A,#0AAH ;load A with AA
08	000D	F590		MOV	P1,A ;send AAH to port 1
09	000F	120300		LCALL	DELAY
10	0012	80EC		SJMP	BACK ;keep doing this
11	0014	;-----this is the delay subroutine			
12	0300		ORG	300H	
13	0300	C004	DELAY:	PUSH	4 ;PUSH R4
14	0302	C005		PUSH	5 ;PUSH R5
15	0304	7CFF		MOV	R4,#0FFH ;R4=FFH
16	0306	7DFF	NEXT:	MOV	R5,#0FFH ;R5=255
17	0308	DDFE	AGAIN:	DJNZ	R5,AGAIN
18	030A	DCFA		DJNZ	R4,NEXT
19	030C	D005		POP	5 ;POP into R5
20	030E	D004		POP	4 ;POP into R4
21	0310	22		RET	;return to caller
22	0311			END	;end of asm file

Solution:

First notice that for the PUSH and POP instructions we must specify the direct address of the register being pushed or popped. Here is the stack frame.

After the first LCALL

0B		
0A		
09	00	PCH
08	0B	PCL

After PUSH 4

0B		
0A	99	R4
09	00	PCH
08	0B	PCL

After PUSH 5

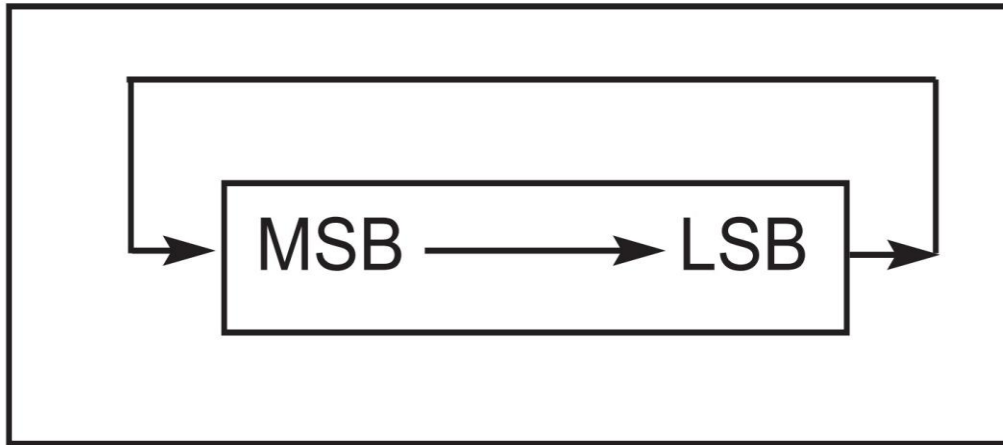
0B	67	R5
0A	99	R4
09	00	PCH
08	0B	PCL

Write code to send 55H to ports P1 and P2, using (a) their names, (b) their addresses.

Solution:

- (a) **MOV A,#55H ;A=55H**
 MOV P1,A ;P1=55H
 MOV P2,A ;P2=55H
- (b) **From Table 5-1, P1 address = 90H; P2 address = A0H**
 MOV A,#55H ;A=55H
 MOV 90H,A ;P1=55H
 MOV 0A0H,A ;P2=55H

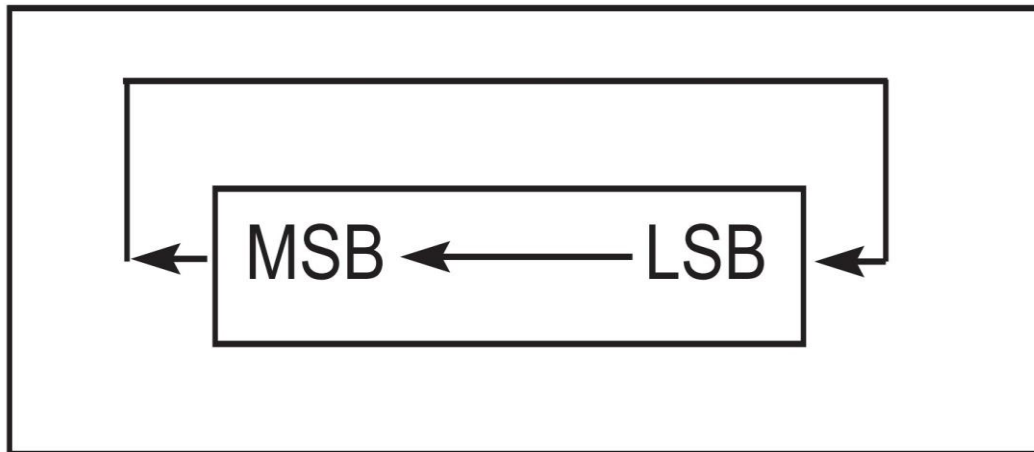
RR A ;rotate right A



Rotate the bits of the accumulator (A) to the right.

The least significant bit (LSB) moves into the most significant bit (MSB), and all other bits are shifted one position to the right.

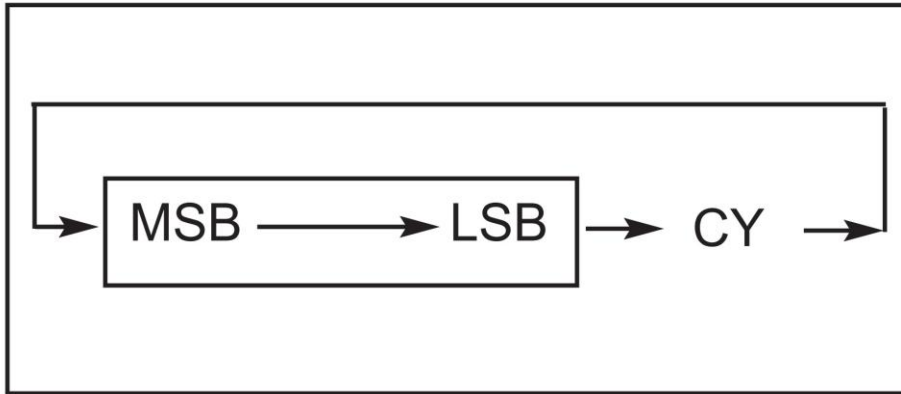
RL A ;rotate left A



Rotate the bits of the accumulator (A) to the left.

The most significant bit (MSB) moves into the least significant bit (LSB), and all other bits are shifted one position to the left.

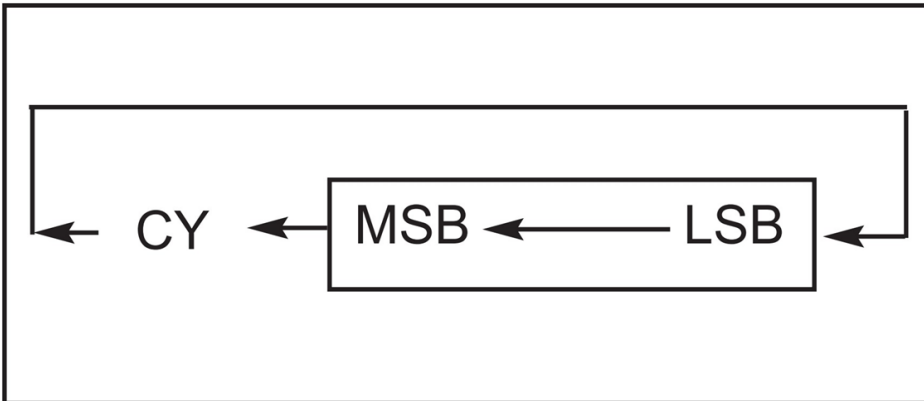
RRC A ;rotate right through carry



The least significant bit (LSB) of the accumulator (bit 0) moves into the carry flag.

The carry flag's previous value moves into the most significant bit (MSB) of the accumulator.

RLC A ;rotate left through carry



The most significant bit (MSB) of the accumulator (bit 7) moves into the carry flag.

The carry flag's previous value moves into the least significant bit (LSB) of the accumulator.

; RR A

; Example: Rotate the contents of A right

ORG 0000H ; Start at address 0000H

MOV A, #96H ; Load accumulator with 10010110B (96H)

RR A ; Rotate A right

END ; End of the program

- A = 10010110B = 96H
- Bit 0 (LSB) of A moves to Bit 7 (MSB).
- The remaining bits shift one position to the right.

Result:

- A = 01001011B (4BH)

; RL A

; Example: Rotate the contents of A left

ORG 0000H ; Start at address 0000H

MOV A, #96H ; Load accumulator with 10010110B (96H)

RL A ; Rotate A left

END ; End of the program

- A = 10010110B (96H)
- Bit 7 (MSB) of A moves to Bit 0 (LSB).
- The remaining bits shift one position to the left.

Result:

- A = 00101101B (2DH)

; RRC A

; Example: Rotate the contents of A right, including carry

ORG 0000H ; Start at address 0000H

MOV A, #96H ; Load accumulator with 10010110B (96H)

SETB C ; Set the carry flag to 1

RRC A ; Rotate A right through the carry flag

END ; End of the program

- A = 10010110B (96H) and C = 1
- Bit 0 (LSB) of A (0) moves to the carry flag (C).
- C (1) moves into bit 7 (MSB) of A.
- The remaining bits in A shift one position to the right.
- A = 11001011B (CBH)
- C = 0

RLC A

; Example: Rotate the contents of A left, including carry

ORG 0000H	; Start at address 0000H
MOV A, #0A5H	; Load accumulator A5H
SETB C	; Set the carry flag to 1
RLC A	; Rotate A left through the carry flag
END	; End of the program

- A = 10100101B (0xA5) and C = 1
- Bit 7 (MSB) of A (1) moves to C.
- C (1) moves into bit 0 (LSB) of A.
- The remaining bits in A shift one position to the left.
- A = 01001011B (0x4B)
- C = 1

16 Bytes of Internal RAM. Note: They are both bit- and byte-accessible.

Byte address									
Bit-addressable locations	7F	General-purpose RAM							
	30								
	2F	7F	7E	7D	7C	7B	7A	79	78
	2E	77	76	75	74	73	72	71	70
	2D	6F	6E	6D	6C	6B	6A	69	68
	2C	67	66	65	64	63	62	61	60
	2B	5F	5E	5D	5C	5B	5A	59	58
	2A	57	56	55	54	53	52	51	50
	29	4F	4E	4D	4C	4B	4A	49	48
	28	47	46	45	44	43	42	41	40
	27	3F	3E	3D	3C	3B	3A	39	38
	26	37	36	35	34	33	32	31	30
	25	2F	2E	2D	2C	2B	2A	29	28
	24	27	26	25	24	23	22	21	20
	23	1F	1E	1D	1C	1B	1A	19	18
	22	17	16	15	14	13	12	11	10
	21	0F	0E	0D	0C	0B	0A	09	08
	20	07	06	05	04	03	02	01	00
	1F	Bank 3							
	18								
17	Bank 2								
10									
0F	Bank 1								
08									
07	Default register bank for R0 - R7								
00									

SFR RAM Address (Byte and Bit)

Byte address	Bit address								
FF									
F0	F7	F6	F5	F4	F3	F2	F1	F0	B
E0	E7	E6	E5	E4	E3	E2	E1	E0	ACC
D0	D7	D6	D5	D4	D3	D2	D1	D0	PSW
B8	--	--	--	BC	BB	BA	B9	B8	IP
B0	B7	B6	B5	B4	B3	B2	B1	B0	P3
A8	AF	--	--	AC	AB	AA	A9	A8	IE
A0	A7	A6	A5	A4	A3	A2	A1	A0	P2
99	not bit-addressable								SBUF
98	9F	9E	9D	9C	9B	9A	99	98	SCON
90	97	96	95	94	93	92	91	90	P1
8D	not bit-addressable								TH1
8C	not bit-addressable								TH0
8B	not bit-addressable								TL1
8A	not bit-addressable								TL0
89	not bit-addressable								TMOD
88	8F	8E	8D	8C	8B	8A	89	88	TCON
87	not bit-addressable								PCON
83	not bit-addressable								DPH
82	not bit-addressable								DPL
81	not bit-addressable								SP
80	87	86	85	84	83	82	81	80	P0

Special Function Registers

Bit Addresses for All Ports

P0	Addr	P1	Addr	P2	Addr	P3	Addr	Port's Bit
P0.0	80	P1.0	90	P2.0	A0	P3.0	B0	D0
P0.1	81	P1.1	91	P2.1	A1	P3.1	B1	D1
P0.2	82	P1.2	92	P2.2	A2	P3.2	B2	D2
P0.3	83	P1.3	93	P2.3	A3	P3.3	B3	D3
P0.4	84	P1.4	94	P2.4	A4	P3.4	B4	D4
P0.5	85	P1.5	95	P2.5	A5	P3.5	B5	D5
P0.6	86	P1.6	96	P2.6	A6	P3.6	B6	D6
P0.7	87	P1.7	97	P2.7	A7	P3.7	B7	D7

Single-Bit Instructions

Instruction		Function
SETB	bit	Set the bit (bit = 1)
CLR	bit	Clear the bit (bit = 0)
CPL	bit	Complement the bit (bit = NOT bit)
JB	bit,target	Jump to target if bit = 1 (jump if bit)
JNB	bit,target	Jump to target if bit = 0 (jump if no bit)
JBC	bit,target	Jump to target if bit = 1, clear bit (jump if bit, then clear)
SETB	C	make CY = 1
CLR	C	clear carry bit (CY = 0)
CPL	C	complement carry bit
MOV	b,C	copy carry status to bit location (CY = b)
MOV	C,b	copy bit location status to carry (b = CY)
JNC	target	jump to target if CY = 0
JC	target	jump to target if CY = 1
ANL	C,bit	AND CY with bit and save it on CY
ANL	C,/bit	AND CY with inverted bit and save it on CY
ORL	C,bit	OR CY with bit and save it on CY
ORL	C,/bit	OR CY with inverted bit and save it on CY

Show how the flag register is affected by the following instructions.

MOV A,#0F5H

;A=F5 hex

ADD A,#0BH

;A=F5+0B=00

Solution:

F5H	1111 0101
+ 0BH	+ 0000 1011
100H	0000 0000

After the addition, register A (destination) contains 00 and the flags are as follows:

CY = 1 since there is a carry out from D7.

P = 0 because the number of 1s is zero (an even number).

AC = 1 since there is a carry from D3 to D4.

Assume that RAM locations 40 - 44 have the following values. Write a program to find the sum of the values. At the end of the program, register A should contain the low byte and R7 the high byte. All values are in hex.

40=(7D)

41=(EB)

42=(C5)

43=(5B)

44=(30)

Solution:

	MOV	R0,#40H	;load pointer
	MOV	R2,#5	;load counter
	CLR	A	;A=0
	MOV	R7,A	;clear R7
AGAIN:	ADD	A,@R0	;add the byte pointer to A by R0
	JNC	NEXT	;if CY=0 don't accumulate carry
	INC	R7	;keep track of carries
NEXT:	INC	R0	;increment pointer
	DJNZ	R2,AGAIN	;repeat until R2 is zero

Write a program to add two 16-bit numbers. The numbers are 3CE7H and 3B8DH. Place the sum in R7 and R6; R6 should have the lower byte.

Solution:

CLR	C	;make CY=0
MOV	A,#0E7H	;load the low byte now A=E7H
ADD	A,#8DH	;add the low byte now A=74H and CY=1
MOV	R6,A	;save the low byte of the sum in R6
MOV	A,#3CH	;load the high byte
ADDC	A,#3BH	;add with the carry
		;3B + 3C + 1 = 78(all in hex)
MOV	R7,A	;save the high byte of the sum

<i>Digit</i>	<i>BCD</i>
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Assume that 5 BCD data items are stored in RAM locations starting at 40H, as shown below. Write a program to find the sum of all the numbers. The result must be in BCD.

40=(71)

41=(11)

42=(65)

43=(59)

44=(37)

Solution:

	MOV	R0,#40H	;load pointer
	MOV	R2,#5	;load counter
	CLR	A	;A=0
	MOV	R7,A	;clear R7
AGAIN:	ADD	A,@R0	;add the byte pointer to A by
R0			
	DA	A	;adjust for BCD
	JNC	NEXT	;if CY=0 don't accumulate
carry			
	INC	R7	;keep track of carries
NEXT:	INC	R0	;increment pointer
	DJNZ	R2,AGAIN	;repeat until R2 is zero

Analyze the following program:

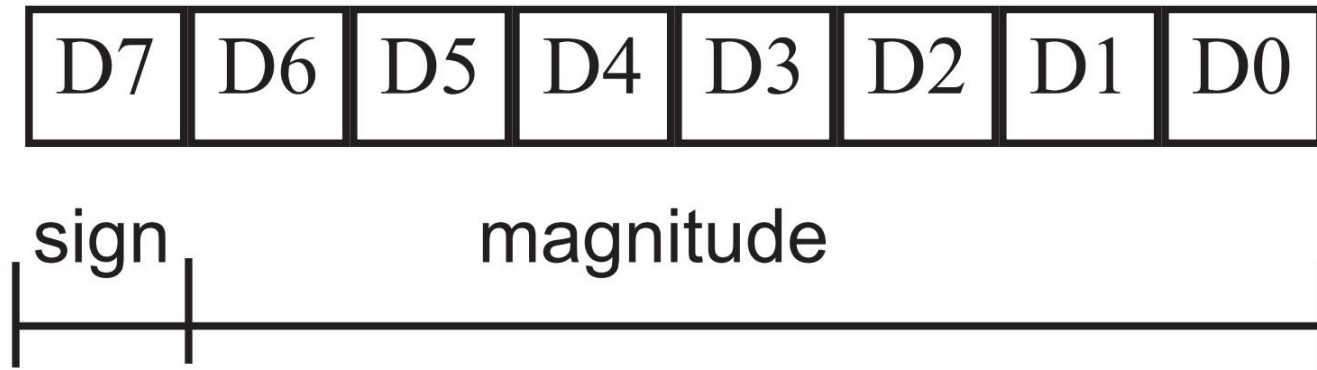
CLR	C	;CY = 0
MOV	A,#62H	;A = 62H
SUBB	A,#96H	;62H - 96H = CCH with CY = 1
MOV	R7,A	;save the result
MOV	A,#27H	;A=27H
SUBB	A,#12H	;27H - 12H - 1 = 14H
MOV	R6,A	;save the result

Solution:

After the SUBB, $A = 62H - 96H = CCH$ and the carry flag is set high indicating there is a borrow.

Since $CY = 1$, when SUBB is executed the second time $A = 27H - 12H - 1 = 14H$. Therefore, we have $2762H - 1296H = 14CCH$.

8-Bit Signed Operand



Show how the 8051 would represent -5.

Solution:

Observe the following steps.

- | | | |
|----|-----------|---------------------------------|
| 1. | 0000 0101 | 5 in 8-bit binary |
| 2. | 1111 1010 | invert each bit |
| 3. | 1111 1011 | add 1 (which becomes FB in hex) |

Therefore -5 = FBH, the signed number representation in 2's complement for -5.

Show how the 8051 would represent -34H.

Solution:

Observe the following steps.

- | | | |
|----|-----------|----------------------------|
| 1. | 0011 0100 | 34H given in binary |
| 2. | 1100 1011 | invert each bit |
| 3. | 1100 1100 | add 1 (which is CC in hex) |

Therefore -34 = CCH, the signed number representation in 2's complement for -34H.

Logical AND Function

Inputs		Output
X	Y	X AND Y
0	0	0
0	1	0
1	0	0
1	1	1



Show the results of the following.

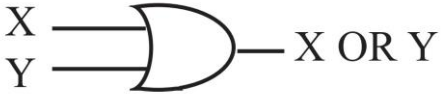
MOV A,#35H ;A = 35H
ANL A,#0FH ;A = A AND 0FH (now A = 05)

Solution:

35H	0011 0101	
<u>0FH</u>	<u>0000 1111</u>	
05H	0000 0101	35H AND 0FH = 05H

Logical OR Function

Inputs		Output
X	Y	X OR Y
0	0	0
0	1	1
1	0	1
1	1	1



Show the results of the following.

MOV A,#04 ;A = 04
ORL A,#30H ;A = A OR 30H (now A = 34H)

Solution:

04H	0000 0100	
<u>30H</u>	<u>0011 0000</u>	
34H	0011 0100	04H OR 30H = 34H

Logical XOR Function

Inputs		Output
X	Y	X XOR Y
0	0	0
0	1	1
1	0	1
1	1	0



Show the results of the following.

MOV A,#54H

XRL A,#78H

Solution:

54H

0101 0100

78H

0111 1000

2CH

0010 1100

54H XOR 78H = 2CH

The XRL instruction can be used to clear the contents of a register by XORing it with itself. Show how “XRL A, A” clears A, assuming that A = 45H.

Solution:

45H

45H

00

0100 0101

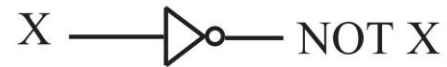
0100 0101

0000 0000

XOR a number with itself = 0

Logical Inverter

Input	Output
X	NOT X
0	1
1	0



Find the 2's complement of the value 85H.

Solution:

MOV A,#85H

CPL A ;1's comp.

ADD A,#1 ;2's comp.

85H = 1000 0101

1'S = 0111 1010

+ 1
—————
0111 1011 = 7BH

Show the code to push R5, R6, and A onto the stack and then pop them back them into R2, R3, and B, where register B = register A, R2 = R6, and R3 = R5.

Solution:

PUSH	05	;push R5 onto stack
PUSH	06	;push R6 onto stack
PUSH	0E0H	;push register A onto stack
POP	0F0H	;pop top of stack into B
		;now register B = register A
POP	02	;pop top of stack into R2
		;now R2 = R6
POP	03	;pop top of stack into R3
		;now R3 = R5

Write a program to copy the value 55H into RAM memory locations 40H to 45H using

(a) direct addressing mode, (b) register indirect addressing mode without a loop, and (c) with a loop.

Solution:

```
(a)  MOV A,#55H    ;load A with value 55H
      MOV 40H,A      ;copy A to RAM location 40H
      MOV 41H,A      ;copy A to RAM location 41H
      MOV 42H,A      ;copy A to RAM location 42H
      MOV 43H,A      ;copy A to RAM location 43H
      MOV 44H,A      ;copy A to RAM location 44H
```

```

(b)   MOV A,#55H      ;load A with value 55H
      MOV R0,#40H     ;load the pointer. R0=40H
      MOV @R0,A       ;copy A to RAM location R0 points to
      INC R0          ;increment pointer. Now R0=41H
      MOV @R0,A       ;copy A to RAM location      R0 points to
      INC R0          ;increment pointer. Now R0=42H
      MOV @R0,A       ;copy A to RAM location      R0 points to
      INC R0          ;increment pointer. Now R0=43H
      MOV @R0,A       ;copy A to RAM location R0 points to
      INC R0          ;increment pointer. Now R0=44H
      MOV @R0,A

```

```

(c)   MOV A,#55      ;A=55H
      MOV R0,#40H    ;load pointer. R0=40H, RAM address
      MOV R2,#05     ;load counter, R2=5
AGAIN: MOV @R0,A     ;copy 55H to RAM location R0 points to
      INC R0         ;increment R0 pointer
      DJNZ R2,AGAIN ;loop until counter = zero

```

Write a program to clear 16 RAM locations starting at RAM address 60H.

Solution:

```
CLR    A           ;A=0
MOV    R1,#60H     ;load pointer. R1=60H
MOV    R7,#16      ;load counter, R7=16 (10 in hex)
AGAIN:MOV @R1,A     ;clear RAM location R1 points to
INC    R1          ;increment R1 pointer
DJNZ   R7,AGAIN    ;loop until counter = zero
```


Write a program to copy a block of 10 bytes of data from RAM locations starting at 35H to RAM locations starting at 60H.

Solution:

```

MOV    R0 , #35H      ;source pointer
MOV    R1 , #60H      ;destination pointer
MOV    R3 , #10       ;counter
BACK:  MOV    A , @R0   ;get a byte from source
        MOV    @R1 , A  ;copy it to destination
        INC    R0       ;increment source pointer
        INC    R1       ;increment destination pointer
        DJNZ   R3 , BACK ;keep doing it for all ten bytes
```

In this program, assume that the word “USA” is burned into ROM locations starting at 200H, and that the program is burned into ROM locations starting at 0. Analyze how the program works and state where “USA” is stored after this program is run.

```
ORG      0000H           ;burn into ROM starting at 0
MOV      DPTR,#200H      ;DPTR=200H look-up table address
CLR      A               ;clear A(A=0)
MOVC     A,@A+DPTR       ;get the char from code space
MOV      R0,A            ;save it in R0
INC      DPTR            ;DPTR=201 pointing to next char
CLR      A               ;clear A(A=0)
MOVC     A,@A+DPTR       ;get the next char
MOV      R1,A            ;save it in R1
INC      DPTR            ;DPTR=202 pointing to next char
CLR      A               ;clear A(A=0)
MOVC     A,@A+DPTR       ;get the next char
MOV      R2,A            ;save it in R2
HERE:    SJMP            HERE ;stay here
;Data is burned into code space starting at 200H
                ORG 200H
MYDATA:    DB "USA"
                END           ;end of program
```

- In the above program ROM locations 200H - 202H have the following contents.

200=('U') 201=('S') 202=('A')

- We start with DPTR = 200H, and A = 0.
- The instruction “MOVC A, @A+DPTR” moves the contents of ROM location 200H (200H + 0 = 200H) to register A.
- Register A contains 55H, the ASCII value for “U”. This is moved to R0.
- Next, DPTR is incremented to make DPTR = 201H. A is set to 0 again to get the contents of the next ROM location 201H, which holds character “S”.
- After this program is run, we have R0 = 55H, R1 = 53H, and R2 = 41H, the ASCII values for the characters “U”, “S” and “A”.

Assuming that ROM space starting at 250H contains “America”, write a program to transfer the bytes into RAM locations starting at 40H.

Solution:

```

      ORG      0000
      MOV      DPTR,#MYDATA      ;load ROM pointer
      MOV      R0,#40H           ;load RAM pointer
      MOV      R2,#7             ;load counter
BACK:  CLR      A                 ;A = 0
      MOVC     A,@A+DPTR         ;move data from code space
      MOV      @R0,A             ;save it in RAM
      INC      DPTR              ;increment ROM pointer
      INC      R0                ;increment RAM pointer
      DJNZ     R2,BACK           ;loop until counter=0
HERE:  SJMP     HERE

;-----On-chip code space used for storing data
      ORG      250H
MYDATA: DB      "AMERICA"
      END
```

Write a program to get the x value from P1 and send x2 to P2, continuously.

Solution:

```

                                ORG      0
                                MOV      DPTR,#300H      ;load look-up table address
                                MOV      A,#0FFH         ;A=FF
                                MOV      P1,A           ;configure P1 as input port
BACK:                          MOV      A,P1           ;get X
                                MOVC     A,@A+DPTR       ;get X squared from table
                                MOV      P2,A           ;issue it to P2
                                SJMP     BACK            ;keep doing it

                                ORG      300H
XSQR_TABLE:                    DB       0,1,4,9,16,25,36,49,64,81
                                END
```

Notice that the first instruction could be replaced with "MOV DPTR,#XSQR_TABLE"

Write a program to toggle P1 a total of 200 times. Use RAM location 32H to hold your counter value instead of registers R0 - R7.

Solution:

```
      MOV     P1, #55H      ;P1=55H
      MOV     32H, #200     ;load counter into RAM loc 32h
LOP1: CPL     P1            ;toggle P1
      ACALL   DELAY
      DJNZ    32H, LOP1     ;repeat 200 times
```

**For each of the following instructions, state to which port the bit belongs.
(a) SETB 86H (b) CLR 87H (c) SETB 92H (d) SETB 0A7H**

Byte address									
Bit-addressable locations	7F	General-purpose RAM							
	30								
	2F								
	2E								
	2D								
	2C								
	2B								
	2A								
	29								
	28								
	27								
	26								
	25								
	24								
	23								
	22								
	21								
	20								
	1F	Bank 3							
	18	Bank 2							
	17	Bank 1							
	10	Bank 1							
	0F	Bank 1							
	08	Bank 1							
	07	Default register bank for R0 - R7							
	00	Default register bank for R0 - R7							

Byte address	Bit address								
FF									
F0	F7	F6	F5	F4	F3	F2	F1	F0	B
E0	E7	E6	E5	E4	E3	E2	E1	E0	ACC
D0	D7	D6	D5	D4	D3	D2	D1	D0	PSW
B8	--	--	--	BC	BB	BA	B9	B8	IP
B0	B7	B6	B5	B4	B3	B2	B1	B0	P3
A8	AF	--	--	AC	AB	AA	A9	A8	IE
A0	A7	A6	A5	A4	A3	A2	A1	A0	P2
99	not bit-addressable								SBUF
98	9F	9E	9D	9C	9B	9A	99	98	SCON
90	97	96	95	94	93	92	91	90	P1
8D	not bit-addressable								TH1
8C	not bit-addressable								TH0
8B	not bit-addressable								TL1
8A	not bit-addressable								TL0
89	not bit-addressable								TMOD
88	8F	8E	8D	8C	8B	8A	89	88	TCON
87	not bit-addressable								PCON
83	not bit-addressable								DPH
82	not bit-addressable								DPL
81	not bit-addressable								SP
80	87	86	85	84	83	82	81	80	P0

Special Function Registers

For each of the following instructions, state to which port the bit belongs.

(a) SETB 86H (b) CLR 87H (c) SETB 92H (d) SETB 0A7H

Solution:

- (a) SETB 86H is for SETB P0.6.
 (b) CLR 87H is for CLR P0.7.
 (c) SETB 92H is for SETB P1.2.
 (d) SETB 0A7H is for SETB P2.7.

Byte address	Bit address								
FF									
F0	F7	F6	F5	F4	F3	F2	F1	F0	B
E0	E7	E6	E5	E4	E3	E2	E1	E0	ACC
D0	D7	D6	D5	D4	D3	D2	D1	D0	PSW
B8	--	--	--	BC	BB	BA	B9	B8	IP
B0	B7	B6	B5	B4	B3	B2	B1	B0	P3
A8	AF	--	--	AC	AB	AA	A9	A8	IE
A0	A7	A6	A5	A4	A3	A2	A1	A0	P2
99	not bit-addressable								SBUF
98	9F	9E	9D	9C	9B	9A	99	98	SCON
90	97	96	95	94	93	92	91	90	P1
8D	not bit-addressable								TH1
8C	not bit-addressable								TH0
8B	not bit-addressable								TL1
8A	not bit-addressable								TL0
89	not bit-addressable								TMOD
88	8F	8E	8D	8C	8B	8A	89	88	TCON
87	not bit-addressable								PCON
83	not bit-addressable								DPH
82	not bit-addressable								DPL
81	not bit-addressable								SP
80	87	86	85	84	83	82	81	80	P0
Special Function Registers									

Write a program to save the Accumulator in R7 of bank 2.

Solution:

SETB	PSW.4
CLR	PSW.3
MOV	R7,A

Write a program to see if the RAM location 37H contains an even value. If so, send it to P2. If not, make it even and then send it to P2.

Solution:

Even means LSB of ACC should be 0, i.e. ACC.0=0

```
MOV    A,37H           ;load RAM location 37H into accumulator
JNB    ACC.0,YES        ;is D0 of reg A 0? if so jump
INC    A               ;it is odd, make it even
YES:   MOV    P2,A       ;send it to P2
```

The status of bits P1.2 and P1.3 of I/O port P1 must be saved before they are changed. Write a program to save the status of P1.2 in bit location 06 and the status of P1.3 in bit location 07.

Solution:

```
CLR      06          ;clear bit address 06
CLR      07          ;clear bit address 07
JNB      P1.2,OVER   ;check bit P1.2,if 0 then jump
SETB     06          ;if P1.2=1,set bit location 06 to 1
OVER:    JNB      P1.3,NEXT ;check bit P1.3, if 0 then jump
         SETB     07          ;if P1.3=1, set bit location 07 to 1
NEXT:    ...
```

Write a program to save the status of bit P1.7 on RAM address bit 05.

Solution:

```
MOV    A,P1.7      ;get bit from port
MOV    05,A         ;save bit
```

Write a program to get the status of bit pin P1.7 and send it to pin P2.0.

Solution:

```
HERE: MOV    A,P1.7      ;get bit from port
      MOV    P2.0,A      ;send bit to port
      SJMP   HERE        ;repeat forever
```

Read and test P1 to see whether it has the value 45H. If it does, send 99H to P2; otherwise, it stays cleared.

Solution:

```
MOV    P2,#00          ;clear P2
MOV    P1,#0FFH        ;make P1 an input port
MOV    R3,#45H         ;R3=45H
MOV    A,P1            ;read P1
XRL    A,R3
JNZ    EXIT            ;jump if A has value other than 0
MOV    P2,#99H
EXIT:...
```

Examine the following code, then answer the following questions.

(a) Will it jump to NEXT?

(b) What is in A after the CJNE instruction is executed?

```
MOV    A,#55H
CJNE   A,#99H,NEXT
...
NEXT:  ...
```

Branch and Looping Instructions
LOOP, JUMP and CALL Instructions

Instruction	Action
JZ	Jump if A = 0
JNZ	Jump if A ≠ 0
DJNZ	Decrement and jump if register ≠ 0
CJNE A, data	Jump if A ≠ data
CJNE reg, #data	Jump if byte ≠ #data
JC	Jump if CY = 1
JNC	Jump if CY = 0
JB	Jump if bit = 1
JNB	Jump if bit = 0
JBC	Jump if bit = 1 and clear bit

Solution:

(a) Yes, it jumps because 55H and 99H are not equal.

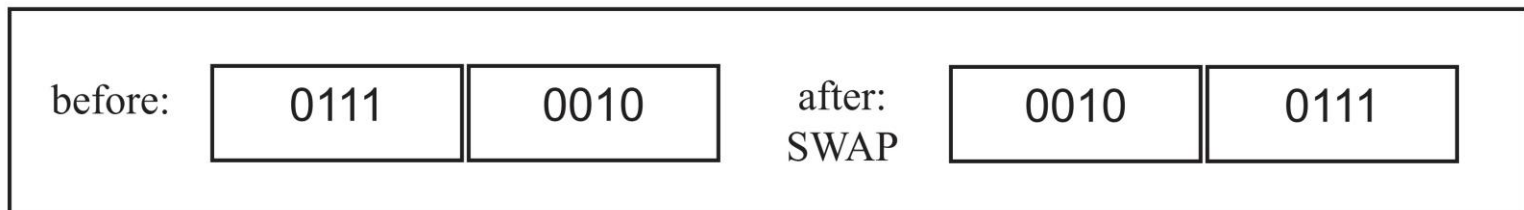
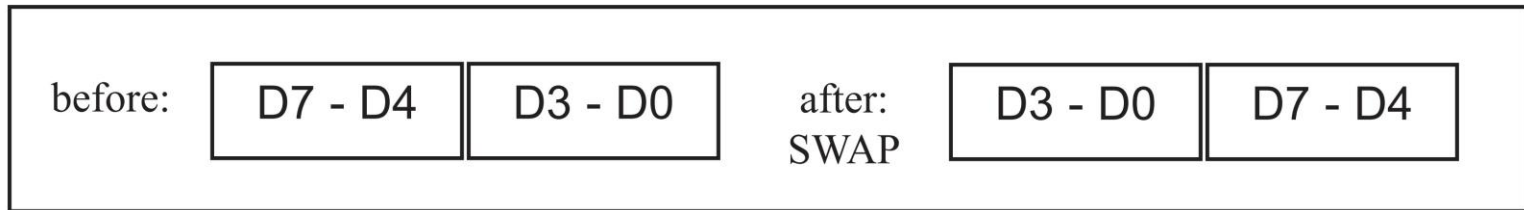
(b) A = 55H, its original value before the comparison.

Write code to determine if register A contains the value 99H. If so, make R1 = FFH; otherwise, make R1 = 0.

Solution:

```
MOV    R1,#0                ;clear R1
CJNE   A,#99H,NEXT          ;if A not equal to 99, then jump
MOV    R1,#0FFH             ;they are equal, make R1 =
FFH
NEXT:...                    ;not equal so R1 = 0
OVER:...                    ;not equal so R1 = 0
```

SWAP A



- (a) Find the contents of register A in the following code.
(b) In the absence of a SWAP instruction, how would you exchange the nibbles?

Write a simple program to show the process.

Solution:

```
(a)      MOV      A, #72H          ;A = 72H
          SWAP     A              ;A = 27H

(b)      MOV      A, #72H          ;A=0111 0010
          RL       A              ;A=1110 0100
          RL       A              ;A=1100 1001
          RL       A              ;A=1001 0011
          RL       A              ;A=0010 0111
```

ASCII Code

- American Standard Code for Information Interchange
- The ASCII (pronounced “ask-E”) code assigns binary patterns for Numbers 0 to 9.
- All the letters of English alphabet, uppercase and lowercase.
- Many control codes and punctuation marks.
- Th

Selected ASCII codes

<i>Hex</i>	<i>Symbol</i>	<i>Hex</i>	<i>Symbol</i>
41	A	61	a
42	B	62	b
43	C	63	c
44	D	64	d
...	...	...	...
59	Y	79	y
5A	Z	7A	z

Dec	Hex	Name	Char	Ctrl-char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	Null	NUL	CTRL-@	32	20	Space	64	40	@	96	60	`
1	1	Start of heading	SOH	CTRL-A	33	21	!	65	41	A	97	61	a
2	2	Start of text	STX	CTRL-B	34	22	"	66	42	B	98	62	b
3	3	End of text	ETX	CTRL-C	35	23	#	67	43	C	99	63	c
4	4	End of xmit	EOT	CTRL-D	36	24	\$	68	44	D	100	64	d
5	5	Enquiry	ENQ	CTRL-E	37	25	%	69	45	E	101	65	e
6	6	Acknowledge	ACK	CTRL-F	38	26	&	70	46	F	102	66	f
7	7	Bell	BEL	CTRL-G	39	27	'	71	47	G	103	67	g
8	8	Backspace	BS	CTRL-H	40	28	(	72	48	H	104	68	h
9	9	Horizontal tab	HT	CTRL-I	41	29	)	73	49	I	105	69	i
10	0A	Line feed	LF	CTRL-J	42	2A	*	74	4A	J	106	6A	j
11	0B	Vertical tab	VT	CTRL-K	43	2B	+	75	4B	K	107	6B	k
12	0C	Form feed	FF	CTRL-L	44	2C	,	76	4C	L	108	6C	l
13	0D	Carriage feed	CR	CTRL-M	45	2D	-	77	4D	M	109	6D	m
14	0E	Shift out	SO	CTRL-N	46	2E	.	78	4E	N	110	6E	n
15	0F	Shift in	SI	CTRL-O	47	2F	/	79	4F	O	111	6F	o
16	10	Data line escape	DLE	CTRL-P	48	30	0	80	50	P	112	70	p
17	11	Device control 1	DC1	CTRL-Q	49	31	1	81	51	Q	113	71	q
18	12	Device control 2	DC2	CTRL-R	50	32	2	82	52	R	114	72	r
19	13	Device control 3	DC3	CTRL-S	51	33	3	83	53	S	115	73	s
20	14	Device control 4	DC4	CTRL-T	52	34	4	84	54	T	116	74	t
21	15	Neg acknowledge	NAK	CTRL-U	53	35	5	85	55	U	117	75	u
22	16	Synchronous idle	SYN	CTRL-V	54	36	6	86	56	V	118	76	v
23	17	End of xmit block	ETB	CTRL-W	55	37	7	87	57	W	119	77	w
24	18	Cancel	CAN	CTRL-X	56	38	8	88	58	X	120	78	x
25	19	End of medium	EM	CTRL-Y	57	39	9	89	59	Y	121	79	y
26	1A	Substitute	SUB	CTRL-Z	58	3A	:	90	5A	Z	122	7A	z
27	1B	Escape	ESC	CTRL-[	59	3B	;	91	5B	[	123	7B	{
28	1C	File separator	FS	CTRL-\	60	3C	<	92	5C	\	124	7C	
29	1D	Group separator	GS	CTRL-]	61	3D	=	93	5D	]	125	7D	}
30	1E	Record separator	RS	CTRL-^	62	3E	>	94	5E	^	126	7E	~
31	1F	Unit separator	US	CTRL-`	63	3F	?	95	5F	`	127	7F	DEL

ASCII Code for Digits 0 - 9

Key	ASCII (hex)	Binary	BCD (unpacked)
0	30	011 0000	0000 0000
1	31	011 0001	0000 0001
2	32	011 0010	0000 0010
3	33	011 0011	0000 0011
4	34	011 0100	0000 0100
5	35	011 0101	0000 0101
6	36	011 0110	0000 0110
7	37	011 0111	0000 0111
8	38	011 1000	0000 1000
9	39	011 1001	0000 1001

For 8051 Microcontroller, interpret the contents of various locations and the position of stack pointer after executing each and every instruction of the following code.

Line Number	Program	Contents	
1.	MOV 00H,#29H	00H :	; SP :
2.	MOV 10 H,#30H	10H :	; SP :
3.	MOV 18H,#31H	18H :	; SP :
4.	MOV SP.#29H	SP :	
5.	PUSH 00H	00H :	; SP :
6.	PUSH 18H	18H :	; SP :
7.	PUSH 30H	30H :	; SP :
8.	POP 31H	31H :	; SP :
9.	POP 18H	18H :	; SP :
10.	PUSH 18H	18H :	; SP :
11.	POP 19H	19H :	; SP :
12.	END		

For 8051 Microcontroller, interpret the contents of various locations and the position of stack pointer after executing each and every instruction of the following code.

Line Number	Program	Contents	
1.	MOV 00H,#29H	00H : 29H	; SP : 07H
2.	MOV 10 H,#30H	10H : 30H	; SP : 07H
3.	MOV 18H,#31H	18H : 31H	; SP : 07H
4.	MOV SP.#29H	SP : 29H	
5.	PUSH 00H	00H : 29H	; SP : 2AH
6.	PUSH 18H	18H : 31H	; SP : 2BH
7.	PUSH 30H	30H : 00H	; SP : 2CH
8.	POP 31H	31H : 00H	; SP : 2BH
9.	POP 18H	18H : 31H	; SP : 2AH
10.	PUSH 18H	18H : 31H	; SP : 2BH
11.	POP 19H	19H : 31H	; SP : 2AH
12.	END		

Examine the size of delay in the following program, if the crystal frequency of 8051 microcontroller is 11.0592 MHz.

Label	Opcode	Machine Cycle
	MOV R0,#0A0h	1
L3:	MOV R1,#40h	1
L2:	MOV R3,#20h	1
L1:	DJNZ R3, L1	2
	DJNZ R1,L2	2
	DJNZ R0,L3	2
	RET	2

- $MC = 1 + 160 + 160 * 64 + 160 * 64 * 32 * 2 + 64 * 160 * 2 + 160 * 2 + 2$
- $MC = 6,86,563$
- $T = 6,86,563 * 1.085 \mu s = 7,44,920.855 \mu s = 744.92 \text{ ms}$

For 8051 Microcontroller, identify the error if any in the following instructions and discuss the error in detail.

- i. MOV R1,R2**
- ii. ADD #40H,A**
- iii. MOV A, @R2**
- iv. XOR R0,23H**
- v. MOV C,A**

- i. No error
- ii. Error: Invalid syntax. ;A must be the destination
- iii. Error: Invalid register usage. ;R0 and R1 only allowed
- iv. Error: Invalid syntax. ;XRL and A should be the source or destination
- v. Error: Invalid instruction. ; Does not support a direct move operation between the accumulator (A) and the carry flag (C).

Analyze the following code and specify the content of various registers and Program Status Word (PSW) after the execution of each instruction.

	ORG 00H		
	MOV A,#00H	A=? PSW=?	00H 00H
	MOV R5,A	R5=?	00H
	MOV R0,#0F4H	R0=?	0F4H
	ADD A,#89H	A=? PSW=?	89H 00H
	JNC N_1		
	INC R5		
N_1 :	ADD A,#0A5H	A=? PSW=?	A=89H+A5H=12EH
	JNC N_2		A = 2EH (Lower 8 bits of 12EH)
	INC R5		
N_2 :	ADD A,#0E2H		PSW = 81H (Carry flag C = 1, Auxiliary Carry AC = 1)
	JNC OVER		
	INC R5		
OVER :	MOV @R0,A		
	END		

The initial values of the memory location (06h-09h) are given below in the Table

Memory location	Initial value	Final value	
09h	88(h)	?	88H
08h	10(h)	?	67H
07h	AD(h)	?	ADH
06h	67(h)	?	88H

Identify the final values in these locations after execution of the following program.

ORG 00H

PUSH 06H

MOV SP, #09H

POP 06H

END

- PUSH 06H ; SP=08H, Stack contents: 08H=67H
- MOV SP, #09H ; SP=09H
- POP 06H ; Value Popped: 09H=88H; Result:06H=88H SP=08H

Identify the addressing mode for

a) POP 0 b) MOV A, #255 c) MUL AB d) 40H, #40H e) MOV @R1, A

- a. Direct Addressing
- b. Immediate Addressing
- c. Register Addressing
- d. Invalid ; MOV 40H, #40H is Direct Addressing
- e. Indirect Addressing

Sketch the content of the stack after the execution of the following code:

PC	Opcode	Mnemonic	Operand
000B	120300	LCALL	DELAY
000E	80F0	SJMP	BACK
0010 ; this is the delay subroutine			
0300		ORG	300H
0300		DELAY:	
0300	7DFF	MOV	R5, #0FFH
0302	DDFE	AGAIN:	DJNZ R5, AGAIN
0304	22	RET	

- LCALL DELAY ; 08h: 0Eh, 09h: 00h, sp: 09h
- RET ; sp = 07h

Assume A = 79H, R0 = 94H. what is the content in A after the execution of the following instructions.

- i. MOV A, R0**
- ii. SWAP A**
- iii. XCHD A, R0**
- iv. ADD A, R0**

- i. A = R0 = 94H, R0 = 94H (remains unchanged)
- ii. A = 49H, R0 = 94H (remains unchanged)
- iii. Exchange the low nibble of A with the low nibble of R0.
 A = 44H, R0 = 99H
- iv. A = 44H+99H=DDH

Describe binary addition using 8051 Microcontroller for the data 0FH and F0H and show the status of PSW.

- 00001111 (0FH)

+ 11110000 (F0H)

11111111 (FFH)

- MOV A, #0FH ; Load 0FH into the accumulator (A)
- ADD A, #F0H ; Add F0H to A
- C=0; AC=0; OV=0; P=0

Assume string of data is stored in code space starting at address 200H as shown below MYDATA: “VIT UNIVERSITY”. Write an 8051 ALP to transfer this string of data in reverse order i.e. “YTISREVINU TIV” to RAM locations inside the CPU starting at 40H

ORG 0H

MOV DPTR, #200H ; Load the address of MYDATA into DPTR

MOV R0, #40H ; Initialize R0 (used as RAM pointer) to 40H

MOV R1, #14 ; Initialize R1 with the length of the string

; Calculate the address of the last character in the string

MOV DPTR, #20DH ; DPTR now points to the last byte of the string

REVERSE_LOOP: MOVC A, @A+DPTR ; Read a byte from code space (using DPTR)

MOV @R0, A ; Store the byte in RAM at the address pointed by R0

INC R0 ; Move to the next location in RAM

DEC DPTR ; Move DPTR backward to the previous byte in code space

DJNZ R1, REVERSE_LOOP ; Continue the loop until all 15 bytes are transferred

END

